Agent Harness: Foundations, Initialization, and Self-Evolution

A Reference Architecture for Execution, Memory, and Model–Harness Learning

Author details to be supplied by the responsible researcher

Position Paper / Technical Survey / Reference-Standard Proposal

October 21, 2026

Literature reviewed through October 21, 2026

Abstract

An agent harness makes a language model operational: it assembles context, exposes authorized actions, collects environment observations, evaluates progress, updates state, and decides whether to repeat or terminate. We investigate a specific architectural hypothesis: *if harness component contracts and execution semantics are standardized while their implementations remain task-specific, can an initially executable harness become a reusable substrate for verifiable self-evolution and language-model learning?* We distinguish three stages—*stable component and lifecycle contracts*, *task-specific harness initialization*, and *feedback-driven evolution*—and separate the per-episode execution loop from the slower loop that modifies harness configurations. Our proposed Agent Harness Contract (AHC) defines a normative Core and optional Initialization, Memory, Training, Replay, Optimization, and Safety profiles. We specify a task declaration, initializer, five bootstrap-readiness gates, versioned module manifests, diagnosis-guided editing, independent validation and rollback. We analyze existing runtime SDKs, environment and telemetry protocols, agentic RL frameworks, and harness-optimization studies, identifying complementary strengths without claiming that we introduce the first agent loop or self-improving harness. We relate initial harness quality to reachable optimization paths, separate changes to episodic memory from changes to memory policy, and formalize fixed-harness LLM training, harness-only improvement, and guarded joint optimization. Long-context management, coding, and recommendation with delayed and confounded feedback provide stress tests. Finally, we propose conformance tests and seven falsifiable empirical studies, including initialization ablations and held-out evolution. This is a *position and reference-standard proposal*, not an implemented standard or evidence that standardized modules guarantee autonomous improvement or universal task coverage.

Keywords: Agent harness; reference architecture; task-specific initialization; self-evolution; agentic learning; execution contract; context engineering; model–harness optimization; interoperability.
Research status: Author-originated theses, grounded technical synthesis, proposed standard and falsifiable experiments; no newly measured results.

Contents

1 Introduction **4**

2 The Central Thesis: A Harness Is a Reusable Learning Substrate **4**
2.1 A useful definition of *framework completeness* 5

3 Scope, Terminology, and Prior Art **6**
3.1 What counts as an agent harness? 6
3.2 What the literature already establishes 6

4 Framework Landscape: Where the Existing Pieces Fit and Fail **7**
4.1 Runtime SDKs already implement much of the proposed execution cycle 7
4.2 Environment and telemetry standards solve different halves 8
4.3 Training frameworks are close antecedents, not missing prerequisites 8
4.4 Harness optimization and policy optimization now demonstrably overlap 8
4.5 What precisely is still open? 8
4.6 Three distinct meanings of standardization 9

5 A Formal Model of Agent, Harness, and Environment **9**

6 Components: Minimal Core Versus Optional Capabilities **9**
6.1 Mapping Harness^{2}'s eight components onto the execution contract 10

7 Task-Specific Initialization: From a Common Contract to an Executable Harness **10**
7.1 The separation of stable contracts and task-specific implementations 10
7.2 Bootstrap readiness is not task correctness 11
7.3 Initialization quality limits the reachable improvement path 13
7.4 Two timescales: the execution loop and the evolution loop 13
7.5 Diagnostic credit assignment before selecting an editable module 13
7.6 Worked instantiations: same semantics, different solutions 13
7.7 The boundary between evolution and adaptation 14

8 The Agent Harness Contract (AHC) **14**
8.1 Invariant I: explicit context and action provenance 14
8.2 Invariant II: typed environment effects 14
8.3 Invariant III: completion, correctness, and reward are separate 14
8.4 Invariant IV: working context differs from durable memory 15
8.5 Invariant V: replayable, versioned trajectories 15
8.6 Invariant VI: explicit initialization and promotion boundaries 15
8.7 Minimal lifecycle and event types 15
8.8 How AHC relates to existing standards 15

9 AHC as a Candidate Reference Standard, Not Another SDK **15**
9.1 The six required interfaces 16
9.2 Conformance profiles rather than a rigid ladder 16
9.3 Canonical action lifecycle and safety invariants 17
9.4 Minimal typed data entities 17
9.5 Required conformance tests 17
9.6 Compositionality and expressiveness limits 18
9.7 Adoption and governance 18

10 Agentic Post-Training under a Fixed Harness **18**
10.1 From execution logs to valid learning samples 18
10.2 Transferable skills are a hypothesis, not a consequence 19

11 Long Context and Unified Memory: A Stress Test of a Universal Harness **19**
11.1 Four different information stores 19
11.2 An optional unified *Atom* schema for user, creator, and system memories 20
11.3 Learning context policy without confusing downstream signals 20
11.4 Memory updates as part of diagnosis 20

12 Reward Is an Evidence Pipeline, Not a Single Number **21**

13 Harness Optimization: What Can Be Changed? **21**
13.1 Context and long-horizon memory as harness policies 22

14 Joint Harness–Policy Optimization **22**
14.1 Alternating updates, conservatively . . . 22
14.2 Separating effects with a factorial design . . . 23
14.3 Training–serving consistency and stale trajectories . . . 23

15 A Two-Timescale Learning Architecture **23**
15.1 Four optimization surfaces, not only two . . . 24
15.2 When to optimize which surface . . . 24
15.3 Guarded optimization loop . . . 25
15.4 Training-serving alignment after a harness update . . . 25
15.5 A falsifiable benefit of dual optimization . . . 25

16 What Does "Standardize a Harness" Mean in Practice? **26**
16.1 Conformance is modular rather than a mandatory monolith . . . 26
16.2 Compatibility versus semantic equivalence . . . 26
16.3 Is universal problem representation feasible? . . . 26

17 Cross-Domain Case Studies and Requirements **26**
17.1 Coding and software maintenance . . . 26
17.2 Browser and enterprise workflows . . . 26
17.3 Recommendation and long-term personalization . . . 27

18 A Reference Benchmark for Harness Standardization **27**
18.1 Study A: expressive coverage and implementation cost . . . 27
18.2 Study B: do standardized trajectories help post-training? . . . 28
18.3 Study C: model–harness causal attribution . . . 28
18.4 Study D: long-context and memory robustness . . . 28
18.5 Study E: absent, noisy, and delayed reward . . . 28
18.6 Study F: generalization of skills, context and harness components . . . 28
18.7 Study G: initialization quality and guarded self-evolution . . . 28
18.8 Minimum statistical and reproducibility requirements . . . 29

19 A Practical Path from Proposal to Community Reference Standard **29**

20 Empirical Protocol: Falsifiable Claims **30**
20.1 Benchmark matrix and controls . . . 30
20.2 Metrics . . . 30
20.3 Data, statistical discipline, and release criteria . . . 31

21 Threats to Validity and Failure Modes **31**

22 Discussion: Where Standardization Should Stop **31**

23 Conclusion **32**

A Illustrative Typed Record (Non-normative) **34**

B Illustrative Bootstrap and Evolution Manifest **34**

C Reference Conformance Test Cases **35**

D Proposed Implementation Sequence **35**

1 Introduction

A language-model agent is not just a model responding to a prompt. It is an executable system that assembles context, proposes actions, interacts with an environment, obtains observations, tests progress, updates information, and repeats. This interaction loop is familiar from ReAct-style agents [24]; our concern is how to make it *portable across tasks, explicit enough to evaluate, and capable of supporting continuous improvement*.

The motivating design intuition has three steps. *First*, standardize a narrow set of harness component contracts and execution semantics, without standardizing the task solutions themselves. *Second*, instantiate these contracts for a target task: initialize the context builder, permitted tools, memory scopes, skills, stop criteria, evaluation signals, and safety policy to obtain an executable initial harness H_0. *Third*, collect execution evidence and use it to diagnose weaknesses, propose edits to named modules, validate edits under controlled conditions, and optionally update the underlying LLM policy. The claim worth testing is not that an initialized harness *must* improve, but that standardized modularity can make its improvement *measurable, bounded and transferable*.

This raises sharper questions than whether every agent needs eight files. Which contracts can be shared across a coding repair, browser transaction, research workflow and recommendation intervention? How should an initializer configure task-specific modules without hiding missing capabilities? What does "ready to evolve" mean in the absence of trusted reward? How do we distinguish a memory update within an episode from a persistent change to the memory *policy*? And when should the system change the harness rather than post-train the language model?

The research landscape already contains important pieces. Agent Lightning and Agent-R1 modularize agent learning [11, 2]; Harness^2 and AHE optimize editable runtime components with frozen policy weights [22, 10]; and Self-HPO, SIA, and SafeEvolve study coupled harness–policy updates [20, 5, 12]. Surveys already organize agent runtime responsibilities [4, 14]. We do not claim priority over these advances. Instead, we propose an *initialization-to-evolution reference architecture* and a cross-runtime Agent Harness Contract (AHC) that specifies the evidence needed to test its benefits.

Our contributions are analytical and prescriptive, not experimental: (i) seven explicit design theses; (ii) a standardized-contract versus task-implementation distinction; (iii) a proposed initializer, readiness gates and guarded evolution operator; (iv) a core and independently selectable conformance profiles; (v) a diagnosis-oriented distinction among harness, model, evaluator and environment changes; and (vi) falsifiable cross-domain and initial-harness-quality experiments. Section 2 introduces the theses; Section 7 makes bootstrap and evolution explicit; Section 9 describes the contract; and subsequent sections examine memory, reward, optimization and empirical validity.

2 The Central Thesis: A Harness Is a Reusable Learning Substrate

Our starting point is a concrete engineering hypothesis, rather than an additional list of agent components: *if a language model is to learn how to operate in many environments, the surrounding execution machinery should expose a stable set of interaction semantics even when individual tasks, tools, memories, and evaluators differ*. The ambitious version is that any digital agent task could be described by composing a finite collection of harness capabilities. We consider this a useful *design target*, not an established universality theorem.

Thesis T1—Execution precedes scalable interactive learning. A minimal executable agent loop must expose a meaningful observation, select an action, observe its executed consequence, decide whether to continue, and maintain state. Once this loop is instrumented, the same runtime may support demonstrations, online exploration, preference data, delayed reward collection, and replay. This does *not* imply that a large unified framework is a prerequisite for model training: narrow-purpose agentic RL can be built with task-specific code. The assertion is about a practical path to portability and reliable scaling, not logical necessity.

Thesis T2—What should generalize is the contract, not the task solution. A coding repair, web transaction, and recommendation intervention differ radically in action preconditions, hidden state, and quality measurement. We therefore hypothesize that the appropriate invariant is a lifecycle interface that guarantees *what the model saw, what it proposed, what the environment actually did, what evidence was returned, how state changed, and what outcome signals are available*. A universal eight-component directory would be less portable than this contract. In particular, a memory-free coding task must remain representable without inventing artificial persistent memories.

Thesis T3—Task-specific initialization is the bridge from a standard to an agent. Standard interfaces do not select useful tools, provide correct evidence, or define feedback for a particular problem. We propose an initializer that selects and configures the necessary components under a task declaration, producing a versioned H_0 and an explicit readiness report. Initialization quality may bound which improvements are reachable under finite budget and permissions. A conformant but unusable initial harness is a legitimate negative result.

Thesis T4—The model and harness are distinct learning targets. The policy parameters θ encode reusable decision behavior, whereas harness parameters ϕ define context-selection, action affordance, orchestration, memory, verification, and runtime policies. Harness optimization can produce gains without weight updates; policy training can produce gains without harness changes. Combined optimization introduces interaction effects and potentially harmful non-stationarity. A reference standard must preserve the identity and version of *both* optimization targets.

Thesis T5—Context is not passive input. Context assembly, memory retrieval, compression, evidence ranking, and persistent-memory writes determine the effective information state seen by the model. They are integral to performance and should be treated as testable and possibly learnable policies. For long-horizon tasks, a standard that records only model messages and tool responses would omit decisive causes of success and failure.

Thesis T6—Feedback does not require a universal scalar reward. Some tasks have executable verifiers, some have provisional judge assessments, some have only delayed and policy-dependent user signals, and others have no trustworthy outcome label at all. The common abstraction should be a typed *evaluation event* with maturity, source, uncertainty, and causal linkage; scalar reward is one optional view of that event. A synthetic scalar supplied by a model judge must not silently become ground truth.

Thesis T7—Standardization must be evaluated as a claim. A proposed contract earns the status of a reference standard only if independently built runtimes can conform to it, external tasks can be adapted at acceptable cost, trajectories remain valid for specific training algorithms, and user-facing outcomes improve or costs fall relative to credible alternatives. Thus this paper specifies tests that can falsify the benefits of standardization.

2.1 A useful definition of *framework completeness*

We separately call an instantiated harness *bootstrap ready* if it passes the readiness gates of Section 7; readiness does not imply success. We call a harness *functionally expressive* for a task family if an adapter can represent the required observations, permissible decisions, environment effects, intermediate state, evaluator signals, and lifecycle transitions without hiding task-relevant information. We call it *learning complete for an algorithm* if its records additionally contain the quantities that algorithm needs (for example behavior-policy likelihoods for some off-policy estimators). These are distinct properties: perfect execution support need not imply valid RL data, and valid training records need not imply successful task solving.

Table 1: Explicit research questions derived from the proposed theses. These are open questions, not established results.

	Question	What would count as evidence?
RQ1	Can heterogeneous digital tasks share one formal execution lifecycle?	Adapter coverage, expressive failures, and conformance behavior
RQ2	Are eight named harness components sufficient, necessary, or merely convenient?	Component-need audits across domain families
RQ3	Can task-specific initialization create an executable, auditable H_0 from common contracts?	Bootstrap conformance, missing-capability detection, initialization effort
RQ4	Does a versioned interface materially simplify agentic post-training?	Trace eligibility, integration effort, rollout/train agreement
RQ5	When should the system optimize harness rules versus policy weights?	Matched-budget evolution and 2×2 pairing tests
RQ6	Can context selection and memory policies improve using imperfect feedback?	Independent probes, evidence retention, delayed-outcome controls
RQ7	Do initialized harness edits generalize across new tasks, tools and models?	Held-out initialization/evolution transfer and negative-transfer tests

The strongest claim—one finite framework covers *every problem*—is not defensible without restricting the universe of problems. We instead target *practical universality across discrete, observable or partially observable digital workflows*, with domain-specific typed extensions. This formulation preserves the motivating ambition while stating where it might fail: irreversibility, continuous control, concurrent actors, and rewards without identifiable causal support.

3 Scope, Terminology, and Prior Art

3.1 What counts as an agent harness?

We use *harness* to mean the runtime infrastructure and configuration that turns a model's decisions into bounded interaction with an external task environment. It includes mechanisms for transforming accessible state into a model-readable context, constraining actions, routing tool calls, retaining memory, orchestrating repeated or concurrent steps, collecting evidence, and enforcing resource or safety limits. A model's weights are *not* part of the harness, although smaller learned routers or critics may be embedded in particular modules. The task environment is likewise distinct: its state, transition dynamics, user preferences, and external consequences are not merely agent configuration.

A distinction is essential: a universal *interface* is not a universal *policy*, and a universal functional vocabulary is not a sufficient representation of every possible task. We focus on long-horizon, tool-using or externally stateful LLM systems; continuous low-level control, real-time robotics, multi-party strategic interaction, and domains requiring certified guarantees may need specialized extensions.

3.2 What the literature already establishes

ReAct introduced an influential interaction pattern, and AgentGym studied diverse agent training environments [24, 21]. Agent Lightning and Agent-R1 specifically address modular agent learning [11, 2]. Harness^2 adopts the AHE file-level decomposition and extends it to eight editable components [10, 22]; Guo et al. synthesize six runtime responsibilities (observation, context, control, action, state, verification) [4]. Thus the paper cannot claim a first eight-component taxonomy, a first agent execution loop, or first harness–policy co-optimization. Detailed system-by-system correspondences and remaining interoperability boundaries are analyzed in Section 4.

Table 2: Closest matches and their boundaries (checked against paper abstracts and official documentation as of October 21, 2026). "Gap" means an interface requirement not established by the cited primary purpose; implementations may offer extensions.

Work / framework	Strongest match to our thesis	Boundary relative to a cross-domain learning standard
ReAct [24]	Foundational thought/action/observation loop	Not a typed execution, memory, reward, or rollout provenance specification
LangGraph [9]	Durable graph state, checkpoints, interrupts, persistent stores	General orchestration; RL-ready decision masks, policy provenance, and reward semantics are not its defining contract
OpenAI Agents SDK [15]	Managed loop, tools, handoffs, sessions, tracing, guardrails	Runtime primitives and traces do not by themselves specify cross-framework reward and counterfactual-training semantics
Google ADK [3]	Agents, tools, sessions/memory, extensible integrations	Rich runtime, but a framework-neutral RL trace and harness-policy attribution protocol requires an adapter
MCP [13]	Standardized tools/resources/prompts integration	Does not specify task termination, training rewards, replay validity, or memory-update learning
OpenEnv [6]	Shared environment reset/step/state surface	Environment compatibility is not a full specification of model-visible context and versioned harness decisions
OpenTelemetry GenAI [16]	Standardized operational traces, spans and metrics	Telemetry alone does not guarantee learner-valid action likelihoods or outcome attribution
Agent Lightning [11]	Training/execution separation and RL transition extraction	Does not require identical harness component semantics across all agent implementations
Agent-R1 [2]	Step-level agentic RL, flexible context and layered interfaces	Training architecture is a close antecedent; normative cross-runtime execution and conformance remain a separate proposal
AHE / Harness^2 [10, 22]	Editable components, evidence-driven harness improvement	Freeze base model during harness updates; not a general training-data contract
Self-HPO / SIA [20, 5]	Harness and weights are joint improvement targets	Strong antecedents to co-optimization; evidence does not establish a universal interoperability standard
HarnessCompass / Recuris [27, 25]	Generalization-aware component search / long-horizon memory evolution	Specific optimization mechanisms, not complete task/environment/reward interoperability specs

4 Framework Landscape: Where the Existing Pieces Fit and Fail

A productive reference standard must explain precisely why current systems are not sufficient *in combination*, without falsely claiming they lack capabilities they already implement. We distinguish five existing strata: agent orchestration, environment protocols, execution observability, agent-training frameworks, and harness-evolution algorithms. The gaps below describe compatibility with *our proposed cross-domain learning contract*, not intrinsic defects in the cited projects. Feature sets also change, so Table 2 should be read as a time-stamped qualitative comparison rather than an exhaustive feature audit.

4.1 Runtime SDKs already implement much of the proposed execution cycle

LangGraph separates graph checkpoints from long-term stores and supports durable pause/resume semantics [9]. OpenAI Agents SDK exposes tools, handoffs, guardrails, sessions, and traces [15]; Google ADK offers agents and tools with memory/state facilities [3]. These are concrete engineering evidence that the basic execution-cycle primitives are broadly reusable. A report that merely adds *Context*, *Act*, *Observe*, *Repeat* would not constitute a distinctive technical contribution. What remains unsettled is the common machine-readable *meaning* of the recorded transitions: whether a model chose an action, which schema and guardrails constrained it, whether a side effect actually committed, what state and memory changed, whether a completion condition was verified, and how a learning signal was derived. We propose adapters on top of these runtimes rather than rewriting their schedulers.

4.2 Environment and telemetry standards solve different halves

MCP addresses access to tools, prompts, and resources and explicitly provides a client/server interoperability layer [13]. OpenEnv adopts environment-style `reset`, `step`, and `state` APIs to help agents interact with training worlds [6]. OpenTelemetry GenAI conventions describe observability events [16]. These efforts should be *composed*: a standard action may invoke an MCP tool against an OpenEnv-style environment, while an OpenTelemetry trace carries timing and usage. Yet none of these interfaces, considered alone, determines exactly which bytes and evidence references were placed in the LLM context, which proposed action was sampled by which policy checkpoint, or which delayed feedback legitimately trains that decision. AHC defines a semantic join across them rather than competing with their transports.

4.3 Training frameworks are close antecedents, not missing prerequisites

Agent Lightning explicitly disaggregates agent execution from reinforcement learning and supports heterogeneous existing agents [11]. Agent-R1 makes *steps*, rather than one concatenated history, first-class training units and supports flexible context management [2]. Both materially overlap the technical direction of this report. Our difference is not an invented new RL algorithm. It is a proposed *cross-runtime reference contract* with independent conformance tests, provenance for context and memory transformations, action-effect separation, release gates, and task-family transfer criteria. Whether such extra formalism is worth the implementation cost is an empirical question, especially when a single framework already serves one organization well.

4.4 Harness optimization and policy optimization now demonstrably overlap

AHE uses component, experience, and decision observability in harness evolution [10]; Harness^2 adds task-level and improvement-level recursion with a frozen model [22]. Self-HPO alternates harness updates and RL in coding [20]; SIA studies harness and weight updates across several tasks [5]; SafeEvolve explores their interaction for safety [12]. Consequently, *jointly optimizing harness and model* is already a research direction. We focus on the missing methodological problem: version-correct rollouts and independent causal attribution of changes to θ, ϕ, environment, and evaluator. There is strong reason for caution: a matched-budget analysis finds cases where harness evolution fails to beat much simpler search baselines and has limited held-out generalization [19]. A reference standard should make such negative results easy to produce and interpret, not suppress them. In particular, the reviewed work motivates editable harnesses and joint optimization, but does not by itself establish that a task-specific H_0 can be generated correctly from a common module contract, or that its subsequent improvements transfer beyond the tasks used for selection.

4.5 What precisely is still open?

Four recurring deficits motivate AHC. *Initialization fragmentation*: heterogeneous runtimes and training systems differ in how task declarations select initial tools, context policy, memory and evaluation, and often provide no explicit bootstrap-readiness claim or initial-manifest provenance. *Semantic fragmentation*: the same apparent log event can refer to model intention, validated tool command, or confirmed external side effect. *Learning fragmentation*: successful deployment traces may omit sampling distributions, action masks, reward provenance, or exact model-visible context, making them inadequate for particular post-training methods. *Optimization fragmentation*: improvements to memory, tool routing, verification, and weights are often measured in separate experiments without shared lineage and compute controls. None proves a universal-standard vacuum; rather, together they motivate an explicit *intersection contract* that current tools can implement incrementally.

4.6 Three distinct meanings of standardization

Functional standardization names responsibilities (memory, tools, orchestration, verification). **Interface standardization** specifies input/output types, lifecycle semantics, failure modes, and versioning. **Behavioral standardization** would constrain concrete action policies or module implementations. We advocate strong compatibility requirements for the first two and intentionally weak requirements for the third. Different agents should be free to omit persistent memory, select different planners, or use a non-neural evaluator without violating a shared contract.

5 A Formal Model of Agent, Harness, and Environment

Definition 1 (Task environment)**.** *An environment $\mathcal{E}_e$ for task e consists of latent states $s_t \in \mathcal{S}_e$, admissible environment actions $u_t \in \mathcal{A}_e(s_t)$, observations $o_t \in \mathcal{O}_e$, a transition kernel $P_e(s_{t+1}, o_{t+1} \mid s_t, u_t)$, a potentially delayed reward or utility mechanism R_e, and an episode-completion predicate D_e. The observed completion condition need not equal user satisfaction or a training reward.*

Definition 2 (Agent harness)**.** *A versioned harness $\mathcal{H}_\phi$ specifies a context assembly function B_ϕ, an action interface/adapter A_ϕ, an execution-control function X_ϕ, a working-state update U_ϕ, an evaluator V_ϕ, a termination policy D_ϕ, and an instrumentation function L_ϕ. Components may be deterministic, stochastic, or independently trainable. The parameters ϕ include configuration, prompts, skill libraries, retrieval and routing rules, executable code, and versioned assets.*

Definition 3 (LLM decision policy)**.** *The LLM policy $\pi_\theta(a_t \mid c_t, \mathcal{A}_t^{\text{allowed}})$ maps a model-visible context c_t and currently allowed actions to a decision a_t, which can include a tool call, final answer, delegation, request for more information, or a constrained memory operation.*

At step t, the harness first constructs

$$c_t = B_\phi(q, h_t, m_t, o_t, b_t), \quad a_t \sim \pi_\theta(\cdot \mid c_t, \mathcal{A}_t^{\text{allowed}}), \tag{1}$$

where q is the task, h_t is a recorded interaction history, m_t is persistent or retrievable memory, and b_t contains budgets and policy constraints. The action adapter and executor produce an admissible environment action $u_t = A_\phi(a_t)$ and observation o_{t+1}; then

$$(h_{t+1}, m_{t+1}, b_{t+1}) = U_\phi(h_t, m_t, b_t, a_t, o_{t+1}). \tag{2}$$

The evaluator emits structured outcome evidence $v_t = V_\phi(\cdot)$, while termination is governed by $d_t = D_\phi(\cdot)$. Runtime validation or a separate environment-defined D_e may override a requested stop, for example when tests fail or a safety approval is missing. If $d_t = 0$, the cycle continues; otherwise a final record is committed.

Key separation. A context is a *view* of partially observable state, not the environment state itself. Likewise, a model-generated "done" response is neither proof of successful task completion nor the reward. Conflating any two of these leads to flawed training transitions and failure attribution. The formal structure resembles a partially observable decision process, but the harness can alter the observation and action interfaces during learning; the resulting effective decision problem is generally non-stationary.

6 Components: Minimal Core Versus Optional Capabilities

Harness[2] exposes eight editable files: system prompt, guardrails, memory, skills, tool guidance, scripts, subagents, and middleware plugin [22]. These are useful editable surfaces, but files are a representation choice, not an invariant of agentic computation. We instead distinguish a *minimal learning-oriented core* from composable capabilities.

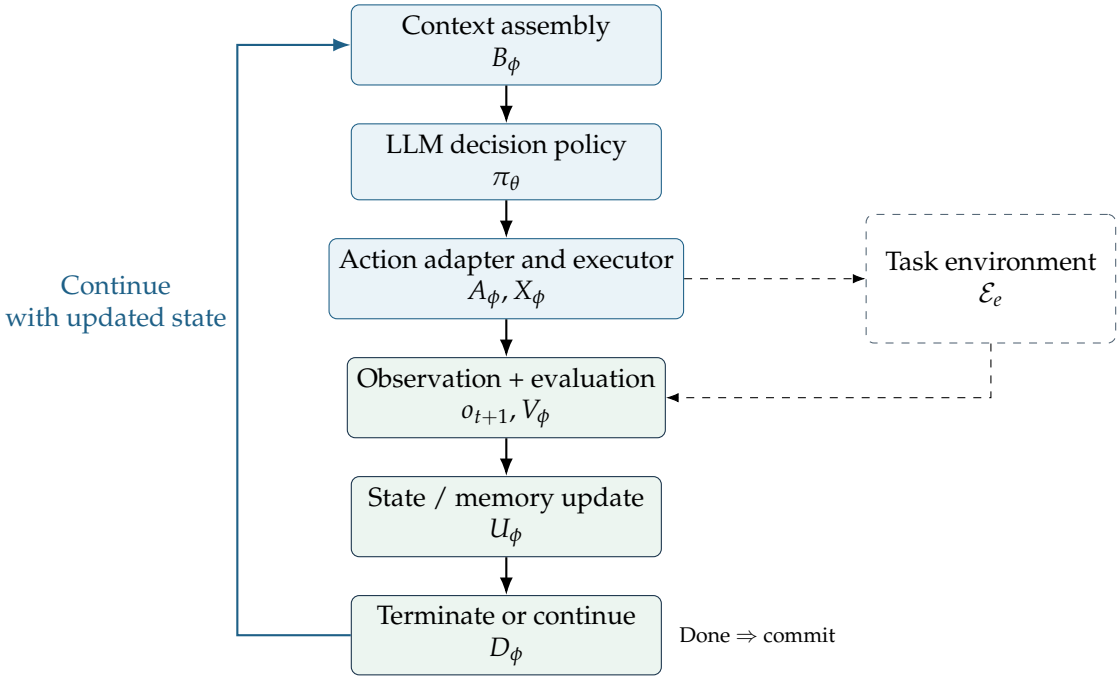


Figure 1: Proposed execution contract. Environment transitions and harness control are distinct; all steps generate versioned records. The evaluator does not necessarily emit a dense reward.

The core is deliberately *behaviorally permissive*. A single-step question-answering agent satisfies it through an identity context builder and no-op updater; a coding agent may use a search tool and sandboxed terminal; a recommendation agent may require delayed observations and contextual bandit estimators; a browser agent may need human approval and transaction rollback. Nested or asynchronous tasks require explicit event linkage rather than a naive linearized string transcript.

6.1 Mapping Harness^2's eight components onto the execution contract

The eight Harness^2 design surfaces are not discarded; they are reinterpreted as optional, composable implementations of a smaller stable interface. Table 4 makes the correspondence explicit. This is our analytical mapping, not a component mapping claimed by the source authors. An implementation MAY merge scripts and tools, embed tool guidance in schema, expose skills as retrievable resources, or use no subagents at all. The contract still records the resulting actions and state transitions. Conversely, the eight surfaces do not by themselves define an evaluator/reward source, immutable rollout manifest, or RL eligibility rules; these are explicit additions in AHC.

7 Task-Specific Initialization: From a Common Contract to an Executable Harness

A stable component interface does not imply that the component contents are transferable. This section formalizes the missing bridge between a *reference harness architecture* and a harness that can solve a particular task. The proposed design is a research hypothesis and an implementation blueprint, not a demonstrated automatic harness generator.

7.1 The separation of stable contracts and task-specific implementations

Let $\mathcal{C} = \{I_1, \ldots, I_K\}$ denote versioned *component contracts*. Each contract specifies input and output types, lifecycle hooks, declared capabilities, constraints, observable side effects, and evidence that a conforming implementation must emit. A task e supplies a declaration

$$Q_e = (g_e, \mathcal{E}_e, \mathcal{A}_e, \mathcal{O}_e, \mathcal{B}_e, \mathcal{P}_e, \mathcal{F}_e), \tag{3}$$

Table 3: Proposed component model. CORE means the interface must exist, not that a dedicated LLM or process must implement it.

Runtime responsibility	Status	Contractual obligation
Context builder	CORE	Return model-visible content with provenance, budget and version
Decision/action interface	CORE	Declare admissible actions, schema, permissions, fallback
Executor/observer	CORE	Commit action results, side effects, timeouts, errors
Evaluator	CORE	Return completion, validation, evidence, and optionally reward
State/context updater	CORE	Update working state; gate durable-memory writes
Termination/controller	CORE	Support terminal, continue, suspended, and aborted states
Trajectory recorder	CORE	Version and link model calls, actions, observations, rewards
Persistent memory and retrieval	OPTIONAL	Retrieval and write policy; lifetimes and ownership
Skills and procedural templates	OPTIONAL	Triggering, applicability, version, execution evidence
Scripts, tools, adapters	OPTIONAL	Typed schema, effects, idempotence where applicable
Subagents and parallelism	OPTIONAL	Context isolation, parent-child spans, aggregation rules
Middleware and guardrails	OPTIONAL	Permission checks, vetoes, approvals, auditing
Trainable router and critic	OPTIONAL	Independent policy ID and optimization provenance

where g_e is the goal; $\mathcal{A}_e$ and $\mathcal{O}_e$ describe available actions and observations; $\mathcal{B}_e$ specifies budgets; $\mathcal{P}_e$ captures permissions, privacy and other policy restrictions; and $\mathcal{F}_e$ describes obtainable completion and evaluation evidence, which may be incomplete, delayed or unavailable. Importantly, Q_e is a *declaration to be checked*, not a guarantee that the goal is achievable.

A task-specific initializer $\mathfrak{I}$ chooses implementations, assets and configurations z_e that respect the common contracts:

$$z_e = \mathfrak{I}(Q_e, \mathcal{C}, \pi_{\theta_0}, D_e^{\text{seed}}); \qquad \mathcal{H}_{e,0} = \text{Instantiate}(\mathcal{C}, z_e), \tag{4}$$

where D_e^{seed} contains optional examples, documentation or synthetic probes. $\mathfrak{I}$ may be handcrafted, template-based, searched, or LLM-assisted; *none* of these initializer algorithms is part of the required standard. The harness artifact $\mathcal{H}_{e,0}$ includes a frozen manifest of the selected module versions and a declaration of unsupported capabilities. This decomposition creates *implementation reuse* without presuming *solution reuse*: two tasks can use the same context interface while supplying entirely different evidence extractors, memory schemas and evaluators.

7.2 Bootstrap readiness is not task correctness

A system should not enter a self-evolution loop merely because all modules have names. We propose five explicit bootstrap gates:

1. **Interface validity:** required modules conform to declared schemas; optional components may be absent or no-ops.
2. **Executable reachability:** there exists at least one permitted action path from a task observation to an outcome event, including a declared behavior for errors and unavailable tools.
3. **Observable evidence:** action attempts, observed outcomes, and relevant context transformations can be attributed to pinned versions; if reward is unavailable, the system declares this rather than inventing a scalar.
4. **Safe containment:** side-effect permissions, sandbox or approval boundaries, budget ceilings, and rollback constraints are explicit.
5. **Improvement testability:** an independent probe set or other defensible validation protocol exists for comparing a proposed edit against $\mathcal{H}_{e,0}$; when it does not, only constrained/manual maintenance is permitted.

Define $\text{Ready}(\mathcal{H}_{e,0}, Q_e) \in \{0, 1\}$ as the conjunction of these checks. This is an *operational readiness* predicate, not a claim that the agent will complete a task successfully. An agent with valid interfaces

Table 4: An eight-to-execution-loop crosswalk. A component can affect multiple stages.

Harness[2] editable surface	Dominant stages affected	Standardization principle
System prompt	Context assembly, model decisions	Pin instructions and their visibility
Memory	Context, state update	Preserve provenance, owner and write policy
Skills	Context, decisions, control flow	Pin invocation trigger and skill version
Tool guidance	Action discovery/selection	Separate descriptions from permissions
Scripts	Action execution, evaluation	Record actual effects and executor version
Subagents	Delegation, observation, update	Causally link parent/child runs
Guardrails	Action validation, termination	Record veto/approval and policy basis
Plugins / middleware	Any lifecycle stage	Declare hook scope and observable changes

Table 5: Stable contract, task-specific initialization, and potentially evolvable decisions. No row implies every task requires the component.

Module contract	Task-specific initial content	Possible later harness improvement
Context assembly	Source adapters, system instructions, token allocation	Evidence selection, ordering, compression, cache thresholds
Memory	Ownership scope, initial store, retrieval/write policy	Deduplication, expiry, conflict resolution, write gating
Skills and planning	Reusable procedures, triggers, decomposition defaults	Trigger rules, decomposition depth, reusable skill extraction
Action and execution	Tool mappings, schemas, validation, retry policy	Routing, parameter defaults, idempotent recovery
Evaluation and stop	Verifiers, feedback maturity, completion criteria	Better test coverage, stop thresholds, judge calibration
Governance and tracing	Permission boundary, recorder, risk budget	Log coverage, audit quality (never silent permission expansion)

may be badly initialized, may lack a necessary tool, or may face an unidentifiable reward. Such failures must remain visible.

Listing 1: Task-specific harness bootstrap (proposed; does not require LLM-generated components).

```
Input: task spec Q, contracts C, base policy theta_0,
      seed documents D, permissions P, validation probes V
selected = resolve_required_capabilities(Q, C)
z_0 = initialize_task_modules(selected, Q, D)
H_0 = instantiate_and_pin(C, z_0)
report = validate_schemas_and_permissions(H_0, Q)
report += dry_run_actions_and_failures(H_0, V)
report += check_trace_provenance_and_feedback_maturity(H_0)
if not bootstrap_gates_pass(report):
   return H_0, "not-ready", report
return H_0, "ready-for-guarded-probes", report
```

7.3 Initialization quality limits the reachable improvement path

The initializer determines which tools, evidence and permissible changes the system initially exposes. A useful conceptual distinction is the *reachable configuration set* $\mathcal{N}^B(\phi_0; \mathcal{P})$, namely harness configurations reachable from ϕ_0 within an optimization budget B and permissions $\mathcal{P}$. A poor initializer may omit a capability that no permissible edit can add. In that case, learning from failures cannot create a missing authorization or inaccessible observation. Conversely, overloading H_0 with every possible tool can increase latency, prompt ambiguity and unsafe affordances. This yields a testable initialization-performance question:

$$J_B(\theta_0, \phi_0) = \max_{\phi \in \mathcal{N}^B(\phi_0; \mathcal{P})} \widehat{J}(\theta_0, \phi), \tag{5}$$

where $\widehat{J}$ is a validation estimate, not a trustworthy population oracle. We do *not* assert monotonic progress or that this maximum can be found efficiently. Comparing a generic baseline, an expert-initialized harness and an LLM-initialized harness at the same search budget is essential before attributing gains to "self-evolution."

7.4 Two timescales: the execution loop and the evolution loop

The inner *execution loop* uses a pinned H_k for a task episode:

$$c_t = B_{\phi_k}(q, h_t, m_t, o_t), \quad a_t \sim \pi_{\theta_k}(\cdot \mid c_t), \quad (o_{t+1}, h_{t+1}, m_{t+1}, v_t) = \mathrm{Step}_{\phi_k}(a_t). \tag{6}$$

Here m_{t+1} can be a *task-state or authorized memory update*; it is not automatically a change to the executable harness configuration ϕ_k. The outer *evolution loop* consumes a collection of versioned traces D_k and proposes edits $\delta\phi_k$ to named modules:

$$\widetilde{\phi}_{k+1} = \mathrm{Propose}(\phi_k, \mathrm{Diagnose}(D_k)); \qquad \phi_{k+1} = \mathrm{Gate}(\widetilde{\phi}_{k+1}, \phi_k; V_k, \mathcal{P}), \tag{7}$$

where V_k is a held-out or independently governed validation source. Gate may return ϕ_k unchanged. Only candidate edits that preserve the standard interfaces and satisfy the declared release constraints may be promoted. In particular, *changing an episode's memory content is not evidence that the harness has learned a better memory policy*. This separation resolves a frequent ambiguity in self-improving agent claims.

7.5 Diagnostic credit assignment before selecting an editable module

A failing episode is not evidence that the system prompt should be rewritten. For example, a missing decisive fact points first to context coverage; correct evidence with an incorrect tool choice points to the model or tool-discovery policy; a valid attempted call with no effect points to the executor; and a completed task scored incorrectly points to the evaluator. These are only *hypotheses*, because multiple causes may be correlated. We propose a hierarchical diagnosis protocol: first check trace completeness and side-effect truth; then localize the earliest falsifiable divergence; then run minimal counterfactual probes under the same θ_k; finally assign candidate edits to the implicated module. This explicitly distinguishes *root-cause diagnosis* from an LLM's free-form reflection.

To avoid degeneracy, the edit action space is restricted by a *contract-preservation rule*: a change to memory retrieval may alter what evidence is chosen, but must not silently widen data access; a tool-routing edit cannot create a new permission; and an evaluator change is calibrated on independent evidence rather than selected for flattering scores. A harness can propose new tools or skills, but their registration requires an explicit external capability/approval transition.

7.6 Worked instantiations: same semantics, different solutions

Coding agent. Q_{code} declares a repository snapshot, accessible test commands, permitted edit paths, a time budget, and verifiable tests. H_0 may start with source search, diff-aware context

assembly, sandbox execution, and a static verifier. If a failure trace reveals omitted dependencies, the harness optimizer may modify source-retrieval policy while holding the model fixed; if adequate source context was present but the model wrote the wrong patch, post-training may instead be the appropriate intervention. A passed local test is not by itself proof against held-out regressions.

Recommendation agent. Q_{rec} declares user-visible history, candidate/item representations, policy constraints, authorized retrieval and analytics tools, privacy boundaries, and delayed engagement/satisfaction signals. H_0 can initialize separate user, creator and system memory scopes through one typed memory contract; no requirement is imposed that all scopes use identical retrieval policies. When results regress, the system may test whether context selection omitted relevant evidence before proposing a ranking-policy change. Observed clicks after a launch are policy-confounded and do not automatically provide a counterfactual reward for harness search. Promotion requires appropriate randomized evidence or defensible off-policy analysis.

7.7 The boundary between evolution and adaptation

Three claims must be kept separate: *(i)* a harness is *evolvable* if its module configurations are explicitly editable and versionable; *(ii)* it is *self-improving on a benchmark* if an automated proposal-and-validation procedure yields reproducible, cost-matched improvement on an independent task distribution; and *(iii)* it is *generalizing* if improvements survive new tasks, tools and environments. Standard module interfaces establish, at best, the conditions for (i). Neither (ii) nor (iii) follows without experiments. This is the central falsifiable claim of the initialization-to-evolution perspective.

8 The Agent Harness Contract (AHC)

We propose AHC as a research-level interoperability contract, not a claim that a formally ratified industry specification exists. The contract is defined by six invariants and a minimal event vocabulary.

8.1 Invariant I: explicit context and action provenance

Before a model samples an action, the runtime MUST record (or securely reference) the prompt/-context version, selected memories and skills, available action schemas, instruction hierarchy, token budget, and any policy constraints. Context can be privacy-preserving: cryptographic hashes and access-controlled references can replace raw sensitive contents in shared traces, while replay access remains permission-bound. A record that only stores the model's final answer is insufficient for reliable post-training.

8.2 Invariant II: typed environment effects

The distinction between *model action* and *executed side effect* is mandatory. The model may propose an unauthorized call, the guardrail may veto it, the tool may time out, or the environment may partially commit a transaction. An action record therefore has proposal, validation, execution, and outcome statuses. Idempotency keys and compensating actions are optional but recommended for external side effects.

8.3 Invariant III: completion, correctness, and reward are separate

Every terminal event declares `reason` (success claimed, verified completion, budget, refusal, user stop, error, or safety stop). Validation evidence is stored independently of a scalar reward. Training rewards can be sparse, delayed, or absent; a judge score is an observation about quality, not automatically the ground truth. This distinction matters in recommendation, where observed

engagement is confounded by exposure policy and may optimize a proxy instead of genuine satisfaction.

8.4 Invariant IV: working context differs from durable memory

The history used to construct the next context may grow while durable user/system memory stays unchanged. Persistent writes require ownership, consent or policy checks, retention rules, provenance, and versioning. Compression and retrieval must preserve references to the evidence they transform. These distinctions make context optimization measurable instead of treating "memory" as one unstructured file.

8.5 Invariant V: replayable, versioned trajectories

Each event includes a trace ID, step ID, parent/correlation ID, task/environment ID, policy checkpoint ID, harness/config hash, schema version, observation/action references, wall-clock/cost fields, and reward provenance where present. Replay may be *exact*, *deterministic sandbox*, or *best-effort external*; the trace MUST identify which guarantee applies. Training records MUST name the action mask and model log-probabilities when these are actually available; no values should be fabricated after execution.

8.6 Invariant VI: explicit initialization and promotion boundaries

An initialized harness has a task declaration, a chosen component manifest, a recorded initializer provenance, and a bootstrap-readiness report. A task episode must pin its harness snapshot; proposed changes to reusable module policies are *candidate revisions*, not in-episode state transitions. A revision can be promoted only through declared validation and authorization gates. The base execution profile may run without automatic evolution; the optional AHC-I profile makes initialization auditable and the AHC-O profile makes promotions auditable.

8.7 Minimal lifecycle and event types

The lifecycle is `declared` $\rightarrow$ `initialized` $\rightarrow$ `ready` $\rightarrow$ `running` $\rightarrow$ (`suspended` | `terminal`), where terminal covers verified success, reported failure, safety stop, cancellation, and exhaustion. Asynchronous action completion and subagent events may create a partial order. For a bare execution-only adapter, earlier initialization events can be summarized in its run manifest. At minimum, the following execution event types exist: `context_assembled`, `model_decision`, `action_validated`, `action_executed`, `observation_received`, `evaluation`, `state_updated`, and `termination`. Appendix A provides an illustrative typed schema; it is deliberately small enough to interoperate with existing tracing rather than replace it.

8.8 How AHC relates to existing standards

MCP standardizes discovery and invocation of tools, resources, and prompts; OpenEnv supplies environment-interaction APIs; OpenTelemetry's GenAI conventions describe observability semantics [13, 6, 16]. They solve complementary slices. AHC adds explicit coupling among the *model-visible decision context*, the *executed environment effect*, the *memory transition*, and *reward/termination provenance* needed by learning pipelines. A deployment should reuse MCP, OpenEnv, or OpenTelemetry primitives rather than invent new wire protocols for already-standardized operations.

9 AHC as a Candidate Reference Standard, Not Another SDK

The proposal is an *Agent Harness Contract* (AHC), a protocol-neutral semantic profile that can be implemented by an existing runtime. Here MUST, SHOULD, and MAY are local normative terms

for this proposed specification, not requirements of an external standards body. AHC separates a small core from capabilities that may be absent, and assumes payloads can be referenced by secure identifiers rather than materialized as plaintext.

9.1 The six required interfaces

Task and environment `I_Task`
Task identifier, goal/evaluation scope, environment version, permissible observation channels, side-effect risk profile, access permissions and budget. MUST distinguish requested outcome from simulator state and from training reward.

Context `I_Context`
Context bundle, exact model-visible message or secure replayable reference, assembly-policy version, included evidence identifiers, omitted evidence metadata where retained, content budget, prompt/tool schema version, and applicable instructions. MUST support audit of *what the policy actually observed*.

Decision `I_Decision`
Action proposal, originating policy checkpoint, tokenizer/sampler identity where relevant, available action namespace and permissions, optional sample likelihoods and token ranges. MUST distinguish policy-selected actions from deterministic or human-selected operations.

Effect `I_Effect`
Validated action, execution attempt, commit/deny/timeout/partial-commit status, observation reference, tool schema and executor version, parent action, permission receipt and retry/idempotency metadata where relevant. MUST not represent an attempted action as a committed result.

Evaluation and termination `I_Eval`
Separate structured records for task-completion claim, verified outcome, judge or heuristic feedback, delayed feedback, reward transformation, terminal reason, and uncertainty. MUST identify whether a reward is observed, estimated, or missing.

State and trace `I_StateTrace`
Working state delta, optional persistent-memory delta, event identifiers, causal parent links, immutable policy/harness/environment/evaluator versions, replay class, and access-control provenance. MUST preserve references to context-transform inputs and outputs when claiming auditable context optimization.

Decision provenance and likelihoods. Model token probabilities are algorithm-specific and may be unavailable from proprietary APIs. AHC therefore does not force implementations to manufacture them: a run can be *execution-conformant* but not *on-policy-RL eligible*. Eligibility must be explicitly declared. A "reward" with no stated target decision, no source, or no maturity status is not valid for the training-conformant profile.

9.2 Conformance profiles rather than a rigid ladder

The previous intuition of levels L0–L4 can accidentally suggest that any sophisticated training engine must support persistent cross-session memory. This would exclude valid memory-free agents. We instead propose independent profiles, with a mandatory base:

This classification distinguishes *data-model conformance* from *learning validity*: an implementation can pass all schema checks and still fail to estimate an unbiased reward. Conversely, a highly effective coding agent might satisfy only Core and Training profiles. No profile imposes a particular vendor, programming language, model architecture, tokenizer, tool transport, or count of harness files.

Table 6: Proposed AHC conformance profiles. Profile membership is not a claim of an existing certification process.

Profile	Dependency	Testable promise
AHC-C (Core)	None	Typed execution loop, causal events, versioned context/effects, terminal reasons
AHC-I (Initialization)	Core	Task manifest, component selection, readiness checks, initializer provenance
AHC-M (Memory)	Core	Owner/scope, durable-write rules, evidence lineage, conflict handling, retention
AHC-T (Training)	Core	Algorithm-qualified decision records, reward provenance, transition extraction
AHC-R (Replay)	Core	Exact/sandbox/best-effort replay declaration and reproducibility test
AHC-O (Optimization)	Core + I and appropriate T/R	Versioned module edits, diagnostic evidence, validation splits and rollback
AHC-S (Safety)	Core	Permissions, action validation, veto/approval events and side-effect audit

9.3 Canonical action lifecycle and safety invariants

For actions with environment effects, the canonical transition stages are

$$a_t^{\text{proposed}} \longrightarrow a_t^{\text{authorized}} \longrightarrow u_t^{\text{attempted}} \longrightarrow \{\text{committed, denied, failed, uncertain}\}. \tag{8}$$

MUST capture whether model action, validator decision, and environment effect agree. A timeout after a payment submission, for example, can leave the commit status unknown; blindly retrying may duplicate an effect. SHOULD support causal and idempotency references. MUST NOT infer authorization solely from the existence of a registered tool. For delegation, parent and child executions MUST carry causal linkage, not an invented total ordering.

9.4 Minimal typed data entities

We propose nine exchange entities: `Task`, `ContextSnapshot`, `Decision`, `ActionAttempt`, `Observation`, `Evaluation`, `MemoryDelta`, `RunManifest`, and `TaskHarnessManifest`. A `TaskHarnessManifest` additionally names the task declaration, selected modules, initializer provenance and bootstrap report (Appendix B). A `RunManifest` pins the tuple

$$\mathcal{V} = \left(v_\theta, v_\phi, v_\mathcal{E}, v_{\text{eval}}, v_{\text{schema}}, v_{\text{tools}}\right). \tag{9}$$

The manifest is *logically immutable*: if any term changes during a run, a new version segment must be recorded. Payloads may be content-addressed and encrypted; a plain hash is not proof that a later user has legal access to decrypt the payload.

9.5 Required conformance tests

A reference implementation should pass deterministic tests for: (i) differences between proposed and executed actions; (ii) denied tool calls; (iii) timeout after possible commit; (iv) terminal success claim that fails an independent verifier; (v) context compaction with an omitted decisive fact; (vi) consent-blocked memory write; (vii) out-of-order delayed reward; (viii) child-agent delegation and join; (ix) harness version change between rollout and training; and (x) mismatched sampling parameters. MUST report the failed test rather than silently downgrade the profile. The included machine-readable example in this report illustrates the shape of the test inputs but does not establish a certified reference implementation. AHC-I should additionally test missing required modules, absent evaluator evidence, unapproved permission expansion and a distinction between memory-content writes and harness-policy revisions.

9.6 Compositionality and expressiveness limits

For task e, let Γ_e map task-local observations and actions into AHC entities. Define *lossless task-relevant expressibility* relative to a specified evaluation functional F_e: if two task histories requiring different valid downstream choices collapse under Γ_e to identical context and state representations, the adapter is insufficient for that task. This is a practical criterion rather than a universal theorem:

$$\Gamma_e(h) = \Gamma_e(h') \;\wedge\; \pi_e^*(h) \neq \pi_e^*(h') \quad \Rightarrow \quad \text{expressive collision.} \tag{10}$$

Here π_e^* denotes an ideal action requirement in the specified task, not an observed learned policy. A protocol cannot guarantee lossless representation of all hidden states; its responsibility is to expose extensions and to make representational collisions measurable. For continuous control, highly concurrent systems, and high-stakes irreversible actions, typed extension profiles are preferable to forcing all semantics into tool-call strings.

9.7 Adoption and governance

The practical route is an open schema plus adapter/conformance test suite, not immediate declaration of an industry standard. We propose a versioned RFC process: stabilize Core and Initialization profiles; solicit two independently written adapters; publish a compatibility report; add optional Training/Memory/Replay profiles; track backward compatibility and data protection; and invite external negative results. Transport and telemetry implementations should reuse MCP, OpenEnv, and OpenTelemetry as appropriate [13, 6, 16]. AHC would become a credible *reference standard* only after actual multi-framework adoption and reproducible interoperability evidence.

10 Agentic Post-Training under a Fixed Harness

Let $e \sim \mathcal{D}$ index an interactive task and τ be the resulting trace under model parameters θ, harness ϕ, and task environment $\mathcal{E}_e$. A simple constrained objective is

$$\begin{aligned} \max_{\theta} \quad & J(\theta;\phi) = \mathbb{E}_{e,\tau\sim p(\tau|e,\theta,\phi)}[R_e(\tau)] \\ \text{s.t.} \quad & \mathbb{E}[C_{\text{tokens}} + \lambda_t C_{\text{time}}] \leq B, \\ & P(\text{unsafe effect}) \leq \epsilon. \end{aligned} \tag{11}$$

The expectation depends on the *entire harness-conditioned trajectory distribution*. Even if the network architecture is held fixed, a new context retrieval policy changes observations, the action mask changes reachable trajectories, and altered tool permissions change which actions can be executed. These are distributional changes, not merely cosmetic prompt edits.

10.1 From execution logs to valid learning samples

A trace is useful for debugging if it records outputs; it is useful for SFT if it additionally identifies trustworthy target actions; it is useful for on-policy RL if it records the actual decision context, action availability, sampling policy and reward assignment. The distinction is summarized below.

The apparent "agent action" may be a model token sequence, a structured tool choice, or a multi-stage decision involving routing and validation. Credit assignment must identify which decisions were produced by trainable parameters and which were injected by deterministic harness logic. Agent Lightning offers a relevant disaggregated architecture and transition extraction; AHC's proposed contribution is to make the interface assumptions explicit at the cross-runtime boundary [11].

Training mode	Additional information required
SFT / imitation	Chosen action or response labels; provenance; eligibility filters; context alignment
Preference training	Comparable candidate outcomes, evaluator assumptions, pair construction
On-policy RL	Policy checkpoint, log probability or reproducible sampler, masks, rewards, terminal state
Off-policy RL	Behavior-policy likelihood/propensity where defined, support overlap, estimators and diagnostics
Harness search	Frozen-model checkpoint, harness diff, matched-task evidence, held-out validation

10.2 Transferable skills are a hypothesis, not a consequence

A standardized trace format may facilitate joint training across domains. It does not imply that a model trained on web-navigation trajectories learns effective recommendation decisions: action semantics, observation noise, reward delays, and safety constraints differ. Transfer must be empirically measured on *held-out task families* and evaluated relative to task-mixture, data-volume, and compute-matched baselines. Transfer can be negative if the policy learns superficial tool names instead of general decision strategies.

11 Long Context and Unified Memory: A Stress Test of a Universal Harness

The most demanding test of a purported generic harness is not whether it supports an arbitrary `memory.md` file, but whether it can maintain useful information across many decisions and tasks without destroying provenance or silently changing the reward problem. Harness^2 defines memory as one editable surface, but does not by itself specify a general retrieval/eviction policy [22]. MemGPT investigates tiered memory [17]; Recuris couples task working state with experience/skill memory for long-horizon tasks [25]. These are complementary algorithmic antecedents. We propose to keep *memory representation*, *memory-selection policy*, and *memory-update policy* separately identifiable in AHC.

11.1 Four different information stores

1. **Raw evidence store**: immutable or versioned documents, tool responses, media descriptors, transactional records, and observation logs. It is the reference layer and may not fit in a model context window.
2. **Working context**: the bounded, model-visible selection and organization of retrieved evidence, task instructions, current environment state, and recent interaction. Context is a *view*, not the canonical memory store.
3. **Episodic execution record**: a causally ordered or partially ordered trajectory of decisions and outcomes, including failed attempts, verification and context transformations.
4. **Persistent memory**: normalized facts, experience, procedures, preferences, and system behavior summaries, selectively written for reuse across executions.

Let I_t be candidate evidence items, z_t the working state, and m_t durable memory. A context-construction policy B_ϕ yields a context c_t subject to a token budget b_t. It may select, summarize, merge, or omit evidence. A separate write policy W_ϕ proposes durable memory changes, but privacy

and validity gates may veto them:

$$c_t = B_\phi(q, z_t, m_t, I_t; b_t), \tag{12}$$

$$\widetilde{\Delta m_t} = W_\phi(q, c_t, a_t, o_{t+1}, v_t), \tag{13}$$

$$m_{t+1} = \mathrm{Commit}_{\mathrm{policy}}(m_t, \widetilde{\Delta m_t}). \tag{14}$$

Compressing context, updating working state, and committing durable memory are three different operations. A successful tool call may change only z_t; a newly discovered preference may justify a persistent write; a denied write must still be a traceable event if a model attempted it.

11.2 An optional unified *Atom* schema for user, creator, and system memories

One practical hypothesis is that several memory owners can share a common atomic representation while differing in access policy and typed payload. Consider a media-centric environment with users, creators (authors), media items, and evolving recommender releases. Rather than building incompatible memory architectures for each owner, propose a record

$$\texttt{Atom} = (\texttt{owner}, \texttt{scope}, \texttt{concept}, \texttt{evidence}, \texttt{statistics}, \texttt{time}, \texttt{context}, \texttt{version}, \texttt{payload}). \tag{15}$$

`concept` may contain normalized term/phrase identifiers, text, and references to multimodal item features. `evidence` identifies underlying media interactions or system events. `statistics` may represent action-conditioned exposure and engagement counts, not merely positive events. `time/context` records recency and setting. `owner/scope` distinguish user, creator, cohort, and system and control read/write authority. `payload` supports typed extensions such as a user interest estimate, creator distribution shift, or launch-policy incident. A common *base record* need not force the same causal interpretation onto all owners.

This is an *illustrative optional memory extension*, not a claim that Atom is the uniquely sufficient representation of knowledge. The important standard-level requirement is to preserve typed owner/scope, timestamps, evidence lineage, update semantics, and revocation ability. Term or phrase normalization is useful for reconciliation and diagnosis, but should not erase source-media references, contradictory evidence, distribution shifts, or the difference between *not engaged* and *never exposed*. A pure embedding-only record may be efficient for similarity search yet insufficient for a human-auditable explanation of why a launch changed a cohort's interests.

11.3 Learning context policy without confusing downstream signals

An informative context policy should preserve task-relevant evidence while minimizing tokens and latency. The optimization target may be task value per token or value under a strict budget, rather than maximal retrieval recall. Compare at least (i) recent-window truncation, (ii) retrieval-only selection, (iii) retrieval plus compression, and (iv) adaptive selection with learned scores. Context selection must be evaluated at matched token budgets and with *evidence omission tests*: deliberately hide a decisive fact and test whether downstream agents detect the missing premise or confidently act anyway. The memory manager must not learn to suppress counterevidence merely because doing so improves a fallible evaluator score.

11.4 Memory updates as part of diagnosis

A useful diagnostic extension links an intervention to changes in different memory scopes. After a retrieval/ranking launch, for example, compare user-interest atoms, creator exposure atoms, and system rollout atoms, while preserving the changed recommendation/exposure policy. Without this linkage, apparent user-interest drift might actually be exposure shift. More generally, causal analysis requires counterfactual or randomized evidence that a textual memory summary does not provide. The harness can record the evidence required for such an analysis, but cannot transform observational correlations into ground truth by naming them "memory."

12 Reward Is an Evidence Pipeline, Not a Single Number

An execution loop can be well-defined even when a true reward is unavailable. We distinguish *progress signals* (what seems to have happened), *task validators* (whether specified conditions were met), *learned judges* (estimated quality), *human or user feedback*, and *causal outcomes* (effects of interventions where identification is warranted). The runtime must not conflate the ability to terminate a loop with the presence of a reliable training target.

Table 7: Reward-evidence taxonomy. The suggested policy is an analytical recommendation, not a published standard.

Signal class	Example	Primary failure mode / mitigation
Verified	Hidden unit tests, exact database predicate	Coverage gaps; hold out tests and check exploitation
Rule-derived	Schema validation, permission check	May detect compliance, not task utility; keep separate
Judged	LLM-as-judge or human rating	Bias, reward hacking, inconsistency; calibrate with independent evidence
Delayed behavioral	Click, return, completion, complaint	Exposure/selection bias, censoring; record policy and time linkage
Preference pair	Two executions compared by annotator	Labeler disagreement and pairing bias; preserve conditions
Absent/unknown	No outcome yet or inaccessible validator	Treat as missing, not zero or failure

For an episode τ, define a vector of typed evidence $\mathbf{y}(\tau)$ and a versioned reward function $g_{\psi,e}$, when appropriate:

$$r_e(\tau;\psi) = g_{\psi,e}(\mathbf{y}(\tau)); \qquad \mathbf{y} = (\text{verification, safety, cost, feedback, delay, uncertainty}). \tag{16}$$

The mapping $g_{\psi,e}$ is explicitly *task-specific* and may not exist. When its weights or evaluator version ψ change, historical scores cannot be compared as if measured on an immutable scale. This is an additional optimization surface beyond θ and ϕ; allowing ψ to drift unconstrained can manufacture apparent improvements.

Long-horizon credit assignment. An outcome may be observed only at episode end. Assigning it to every earlier action is a modeling choice, not direct evidence that each step was useful. AHC should carry causal event links and the eligibility of decisions for credit assignment; the trainer may then use return-based, value-based, process-based, or preference-based methods. Deterministic context assembly and validator vetoes must not be misidentified as trainable LLM actions.

Recommendation as a counterexample to simplistic rewards. Consider recommending media item i to user u under policy μ. Engagement Y is observed only following exposure $E = 1$, and the distribution of shown items depends on μ. Therefore $P(Y \mid E = 1, i, u, \mu)$ is generally not equal to the causal value of intervening to show item i. A trace standard should store exposure propensity when available, delayed-event correlation identifiers, policy/version, and censoring boundaries. A click is neither a universal reward nor a reliable negative label for unexposed items. AHC can facilitate offline evaluation, but unbiased estimation additionally depends on assumptions such as positivity and sufficient confounder measurement.

13 Harness Optimization: What Can Be Changed?

For frozen θ, harness optimization solves

$$\max_{\phi \in \Phi_{\text{safe}}} \; J(\theta,\phi) - \lambda_c \mathbb{E}[C(\theta,\phi)] - \lambda_m \Omega(\phi), \tag{17}$$

where Ω can penalize complexity, unstable components, or change risk. Candidate harness changes include five practically distinct classes:

1. **Context policy:** relevance retrieval, freshness, compression, deduplication, evidence ordering, budget allocation, privacy filtering.
2. **Action affordances:** tool schema descriptions, action search/routing, dynamic tool disclosure, retries, permissions, sandbox placement.
3. **Control flow:** decomposition, subagent assignment, parallel execution, checkpoint/rollback, termination strategy.
4. **Verification:** task-specific tests, rule checkers, reward models, uncertainty handling, human escalation, independent critic.
5. **Memory and experience:** write policies, canonicalization, provenance, expiry, retrieval, reflective lessons, reusable skill extraction.

These changes can be searched by hand, bandits, evolutionary methods, black-box Bayesian optimization, or LLM-based proposal-and-critique. DSPy, TextGrad, and GEPA demonstrate related optimization ideas for modular LM programs and prompts; Harness^2 focuses on a wider range of harness components and experience from recursive editing [8, 26, 1, 22]. Critically, judge-driven harness selection can overfit a noisy evaluator. Evaluator identity, uncertainty, and task selection must therefore be logged and validated on hidden held-out tasks.

13.1 Context and long-horizon memory as harness policies

Long-context management belongs naturally to B_ϕ and U_ϕ, but an eight-part component taxonomy does not specify a retrieval, compression, or memory-retention algorithm. MemGPT already studied tiered memory and explicit control flow [17]; contemporary harness work may incorporate similar primitives. We advocate an explicit separation between *raw evidence*, *working context*, *episodic trace*, and *persistent semantic/procedural memory*. A context policy can select an evidence set S_t under a budget:

$$S_t^* = \arg\max_{S \subseteq \mathcal{I}_t} \widehat{U}_\phi(S \mid q, h_t) \quad \text{s.t.} \quad \sum_{i \in S} \text{tokens}(i) \le B_t. \tag{18}$$

The learned proxy $\widehat{U}_\phi$ can be optimized from downstream task outcomes. But it may reward selecting evidence that persuades the evaluator rather than evidence that is correct. Auditable provenance, conflict indicators, and counterfactual retrieval tests are important.

14 Joint Harness–Policy Optimization

Let $J(\theta, \phi)$ be task utility under the paired model and harness. We distinguish three empirical regimes: *policy-only*, holding ϕ fixed; *harness-only*, holding θ fixed; and *alternating optimization*, updating one while evaluating with the other fixed. Previous work has studied combined updates [5, 12, 20]; our purpose is to identify what an auditable cross-domain protocol would require.

14.1 Alternating updates, conservatively

A generic alternating procedure is

$$\phi_{k+1} \in \text{Select}_{\phi \in \mathcal{N}(\phi_k)} \widehat{J}_{\mathcal{V}_k}(\theta_k, \phi), \tag{19}$$

$$\theta_{k+1} \leftarrow \text{Train}\big(\theta_k; \text{Rollouts}(\theta_k, \phi_{k+1}, \mathcal{T}_k)\big), \tag{20}$$

where $\mathcal{V}_k$ is a held-out harness-selection split and $\mathcal{T}_k$ is training data. Selection must incorporate safety constraints, cost, and confidence intervals; evaluation after both updates should use an *independent* test set. The result is not guaranteed monotonic: the reward can be noisy, the objective non-convex, and the task distribution can drift. A frozen baseline harness, immutable trajectory snapshots, and rollback prevent the newest configuration from silently rewriting the experimental yardstick.

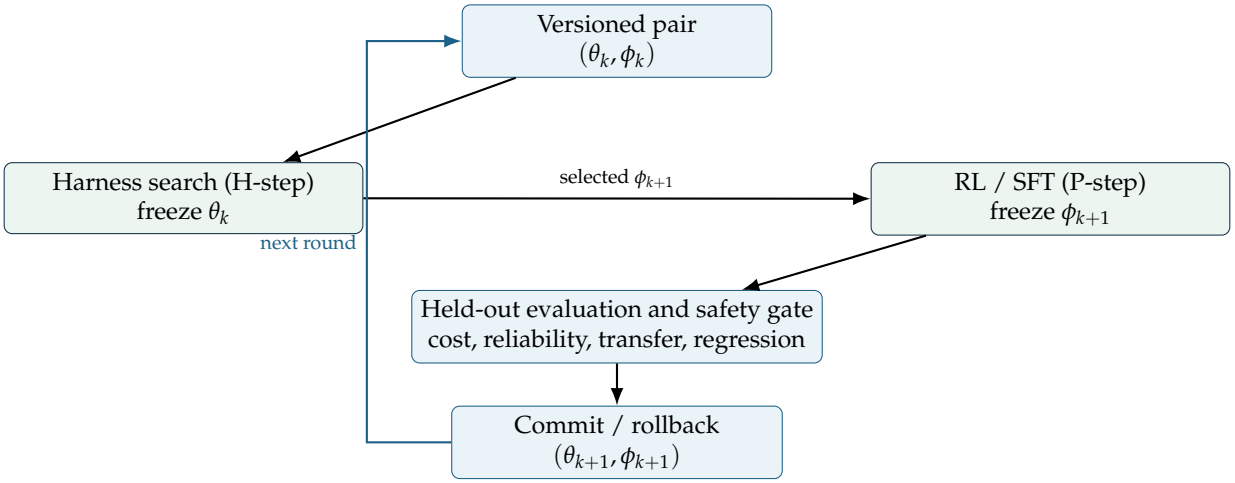


Figure 2: Proposed guarded alternating optimization. Frozen component versions, matched tests, and release gates reduce attribution ambiguity; they do not guarantee convergence.

14.2 Separating effects with a factorial design

Evaluating only the initial and final pair conflates model and harness improvements. On the same task distribution $\mathcal{T}$, evaluate all four pairs:

	Original harness ϕ_0	Updated harness ϕ_1
Original policy θ_0	J_{00}	J_{01}
Updated policy θ_1	J_{10}	J_{11}

Define policy-only gain $J_{10} - J_{00}$, harness-only gain $J_{01} - J_{00}$, and pairwise interaction

$$\Delta_{\text{int}} = J_{11} - J_{10} - J_{01} + J_{00}. \tag{21}$$

A positive Δ_{int} supports synergy *on the specified benchmark and metric*, not universal transfer. Bootstrap confidence intervals should be paired by task; repeated stochastic runs are necessary. A harness that changes which tasks become solvable can affect policy rollouts differently from a harness that simply makes an evaluator more lenient. Distinguishing these effects requires validated outcomes and frozen test conditions.

14.3 Training–serving consistency and stale trajectories

If ϕ changes, previously collected RL trajectories may no longer represent the target policy–harness distribution. PPO/GRPO-style updates require careful accounting of behavior-policy versions and sampling assumptions [18]. We recommend version-stratified rollout buffers and explicit invalidation when context assembly, action masks, or evaluator semantics change. Off-policy reuse requires support overlap or justified corrections, not mere concatenation of every historical trace. For side-effectful environments, counterfactual replay is often impossible; evaluation should use a sandbox or randomized intervention with appropriate authorization.

15 A Two-Timescale Learning Architecture

The distinction between harness optimization and weight optimization matters because they operate on different artifacts, time scales, and evidence. A context-retrieval threshold can change in minutes; an LLM checkpoint may require substantial RL rollouts and review. Some harness choices are differentiable or learned, while many are symbolic policies, executable code, memory records, and access rules. Treating every mutable component as the same parameter class obscures important operational boundaries.

15.1 Four optimization surfaces, not only two

We represent the deployed agent as $\mathcal{G}(\theta, \phi, \psi; \mathcal{E}_e)$, with policy weights θ, harness policies and assets ϕ, and evaluation/reward configuration ψ. The environment $\mathcal{E}_e$ may also evolve externally. To reason about where learning belongs, divide modifications into:

- **Policy adaptation** (θ): learning decisions over tools, planning, self-correction, and evidence use through SFT, preference optimization, RL, or distillation.
- **Harness adaptation** (ϕ): selecting context and skills, changing tool routing, retry policies, execution graphs, retrieval index settings, memory writing, and verification procedures.
- **Evaluator calibration** (ψ): improving reward measurement, judge calibration, hidden-test quality, or delayed-feedback estimators. This may be a separate research objective; it must not be tuned on the same held-out data used to report final gains.
- **Environment adaptation** ($\mathcal{E}_e$): updates to external APIs, user distributions, permissions, task mix, or inventory. These are not controllable harness "improvements" unless explicitly part of the task.

Our design advocates *asymmetric update frequency*: keep the safety-critical kernel and interface schema relatively stable; update low-risk retrieval/context policies more often under offline evaluation; update model checkpoints after collecting valid rollouts; and release new evaluator versions only with recalibration. This is a recommendation, not a proven universally optimal schedule.

15.2 When to optimize which surface

If a newly initialized harness lacks a required capability, do not start a reward-driven editing loop: first repair its task specification or authorization. If the model selected a correct tool but the runtime supplied stale context, prioritize harness retrieval and freshness. If the right tool and necessary evidence were present but the model repeatedly chose the wrong sequence, prioritize policy training. If a transaction committed but the evaluator reports failure, repair outcome instrumentation. If satisfaction changes after a feed-policy launch, first investigate exposure-policy and feedback confounding before modifying policy weights. These are *diagnostic hypotheses*, not automatic diagnoses; matched probes are necessary to distinguish alternatives.

Table 8: Failure localization as a working hypothesis for choosing the optimization target.

Observable failure	Necessary diagnostic probe	Likely intervention if probe confirms cause
Important fact missing	Replay with only the missing evidence restored	Context-selection or memory retrieval rule
Correct evidence, wrong action	Same context with policy alternatives / action audits	SFT/RL and decoding/action policy
Valid proposed action vetoed	Compare declared permissions and tool schema	Guardrail/configuration repair, not blind override
Correct action, faulty effect	Compare tool commit logs and sandbox state	Executor/tool adapter and retry semantics
Premature "done"	Independent verifier against goal predicate	Termination policy, reward/verification design
Evaluator prefers wrong output	Hidden outcome comparison / blinded human audit	Reward-model/evaluator calibration
Performance regresses on new model	Cross-pair old/new model and harness	Harness-policy compatibility review

15.3 Guarded optimization loop

We propose an outer learning process with explicit boundaries. It begins only after the task initializer has produced a manifest and passed the bootstrap readiness gates in Section 7. Candidate harness proposals are generated from a frozen policy version and training-split failure evidence. Proposals undergo static and side-effect checks, followed by matched-cost sandbox rollouts and held-out harness-selection tests. If a candidate survives, new policy rollouts are generated under that *pinned* harness snapshot. Training uses only algorithm-eligible decisions; the updated policy is evaluated with both new and baseline harnesses. Finally an independent release suite checks safety, cost, and transfer before deployment. The four model/harness combinations in Eq. 21 remain mandatory for attribution where feasible.

Listing 2: Proposed reference learning loop; conceptual pseudocode, not measured implementation.

```
initialize policy theta_0, evaluator psi_0
H_0, readiness = task_initialize_and_validate(C, Q, theta_0)
if not readiness: return bootstrap_failure_report
phi_0 = pin(H_0)
for round k:
    freeze(theta_k, psi_0, heldout_test)
    D_k = train_rollouts(theta_k, phi_k)
    failures = diagnose_with_attribution(D_k)
    candidates = propose_contract_preserving_edits(phi_k, failures)
    phi_next = validate_promote_or_keep_current(candidates,
                validation_tasks, theta_k, psi_0)
    pin(phi_next) # failed candidates leave phi unchanged
    traces = collect_train_rollouts(theta_k, phi_next,
                action_masks=True, version_provenance=True)
    eligible = validate_for_training_algorithm(traces)
    theta_next = post_train(theta_k, eligible)
    scores = evaluate_factorial((theta_0, theta_next),
                 (phi_0, phi_next), independent_test, psi_0)
    if safety_gate(scores) and transfer_gate(scores):
        publish(theta_next, phi_next)
    else:
        rollback_to_last_safe_pair()
```

Why the algorithm is not guaranteed to improve. This procedure can fail through proxy reward hacking, model/harness incompatibility, benchmark leakage, high-variance exploration, distribution drift, or insufficient training support. Recent matched-budget comparisons show that simple test-time search can outperform harness evolution in certain settings [19]. Therefore we include random-search, extra-inference, and fixed-harness baselines, and treat cost-normalized generalization as the final criterion.

15.4 Training-serving alignment after a harness update

Suppose a rollout was collected under ϕ_a but training or serving now assembles context with ϕ_b. Even if the string of generated tokens is unchanged, the action distribution is conditioned on different visible evidence and tool masks. Blind reuse of the trajectory for on-policy updates is generally invalid. AHC carries $(\theta, \phi, \psi, \mathcal{E}_e)$ lineage, flags incompatible transitions, and distinguishes direct on-policy replay from importance-corrected off-policy reuse (where likelihood and support assumptions permit). The contract does not solve off-policy learning by itself, but exposes enough metadata to refuse unjustified reuse.

15.5 A falsifiable benefit of dual optimization

Joint optimization is not automatically better than single-axis improvements. Its value should be established only if the interaction term Δ_{int} in Eq. 21 is positive with uncertainty accounted for,

or if the final method delivers a better quality/cost/safety Pareto frontier under equal resources. We recommend reporting the full matrix of policy/harness cross-pairing results, including *negative interactions* where a newly trained policy becomes fragile under the prior harness or a redesigned harness degrades a capable policy.

16 What Does "Standardize a Harness" Mean in Practice?

16.1 Conformance is modular rather than a mandatory monolith

We use the independent Core (C), Initialization (I), Memory (M), Training (T), Replay (R), Optimization (O), and Safety (S) profiles defined in Table 6. This avoids incorrectly requiring persistent cross-session memory for every coding agent, or requiring trainable token probabilities from every proprietary model. A profile claims a *testable interface behavior*, not a specific optimization algorithm. Basic interoperability can improve even when a deployment implements only AHC-C.

16.2 Compatibility versus semantic equivalence

Two harnesses can emit syntactically identical traces while exposing different affordances (e.g., a browser "click" versus a payment "submit"). A usable standard needs capability metadata: action preconditions, idempotence, reversibility, permission level, result visibility, and latency class. *Semantic adapters* can translate between compatible tools, but no adapter can guarantee behavior preservation across arbitrary services. We recommend reference conformance cases with known transitions, including denied actions, timeouts, retries, partial commits, and nested delegation.

16.3 Is universal problem representation feasible?

A broad range of digital tasks can be embedded in the loop of Figure 1. This is a *programming abstraction*, not a theorem that a finite set of high-level components captures all relevant dynamics. For instance: delayed user feedback is not the same as an immediate validation result; partial observability may conceal the true state; real-time actuation may demand non-LLM control; and multi-agent concurrent execution cannot always be reduced to a linear timeline without losing causality. The viable target is therefore *common lifecycle semantics with explicit extensions*, not identical behavior or reward across every domain.

17 Cross-Domain Case Studies and Requirements

17.1 Coding and software maintenance

The environment is a repository and execution sandbox, with actions such as search, edit, compile, and test. Context may include repository map, diffs, stack traces, and code snippets; rewards may combine hidden test pass rates and cost, subject to security constraints. Verification should distinguish finishing the edit from passing a stable hidden test suite. SWE-bench-type tasks and coding benchmarks are natural environments for controlled harness-policy experiments [7, 20]. A reference test asks whether two implementations emit semantically equivalent traces for a failing test, a denied destructive shell command, and a partially applied patch.

17.2 Browser and enterprise workflows

The environment includes pages, forms, user approvals, and external transactional APIs. Action side effects are consequential: submit, refund, cancel, or send are not interchangeable with read-only retrieval. Context construction should preserve the current task objective and permission boundaries; evaluation should compare resulting state with explicit goal conditions and catch unintended edits. τ-bench provides a useful precedent for checking final database state rather than relying only on

conversational plausibility [23]. AHC must preserve asynchronous responses, approvals, and retry idempotence.

17.3 Recommendation and long-term personalization

Here the environment may include a user, creator/author, media inventory, and a recommendation service. The observation stream contains impressions, engagement actions, explicit feedback, and delayed outcomes. A task may request candidate retrieval, explanation, launch diagnosis, or policy adjustment. Working context could combine normalized item descriptors, exposure/action history, cohort-level signals, and system changes. Persistent memory should distinguish user, creator, and system scopes, track timestamped provenance, and avoid representing an unexposed item's non-engagement as a negative observation.

This domain reveals a limitation of generic completion-based RL: a recommendation request can be *served* immediately, whereas value is realized later and only under a particular exposure policy. An illustrative objective may combine satisfaction proxies, safety constraints, novelty, and costs, but a single click metric is not a ground-truth reward. Offline replay for counterfactual ranking may require propensities or doubly robust estimators and adequate support; absent those conditions, observational engagement rates do not identify causal improvement. AHC can record delayed reward links and censoring, but cannot create valid counterfactual evidence by standardization alone.

Table 9: Same contract, different task semantics.

Domain	Key action/observation	Nontrivial evaluation issue
Coding	patch/test → compiler, test result	Hidden test validity; environment determinism
Browser	click/submit → page/-database state	Permissions, irreversibility, long transactions
Recommendation	expose/rerank → delayed engagement	Selection bias, confounding, delayed reward

18 A Reference Benchmark for Harness Standardization

The proposed AHC standard should be assessed using a benchmark whose *unit of analysis is interoperability and trainability*, not only agent task success. We propose *HarnessBench-Protocol* as a name for this *planned evaluation suite*, not an existing dataset or completed implementation. Its experimental units are $(\text{framework}, \text{domain}, \theta, \phi, \psi, \text{seed})$ tuples. It deliberately separates three questions: Can the framework represent the task? Can it be initialized into a valid executable agent? Can its traces train a model correctly? Do optimization gains generalize after holding evidence and budgets constant?

18.1 Study A: expressive coverage and implementation cost

Select three agent runtimes (e.g., LangGraph, OpenAI Agents SDK, and Google ADK) and three task families (coding, browser/enterprise, and recommendation with delayed feedback). Build a native implementation in each runtime and an AHC adapter without modifying the runtime's essential control flow. Use a common test pack of at least: tool validation failure, partial commit, human approval, context compaction, persistence across sessions, child delegation, delayed feedback, and version-switch detection. Report functionality loss, adapter lines of code, engineer-hours, task-specific extensions, and conformance passes. Since implementation cost depends on developer expertise, randomize the order of adaptations or use paired implementers when practical. Falsification: the reference contract adds measurable burden without improved interoperability or auditability.

18.2 Study B: do standardized trajectories help post-training?

For each domain, record three data products from the *same rollouts*: raw conversational/tool transcripts, a framework-native trace, and AHC-typed events. Control the *information content*: a schema cannot legitimately claim to add knowledge absent from a raw source; its proposed benefit is preserving and organizing existing evidence. Evaluate (i) completeness of algorithm-required quantities, (ii) number of excluded or mislabeled training decisions, (iii) reproducibility of context and action masks, and (iv) downstream performance when trained with a matched RL/SFT recipe. An adapter that only adds metadata but not task-relevant evidence should not be credited with a causal improvement in model intelligence. Pre-register the algorithm's minimum trace eligibility rules.

18.3 Study C: model–harness causal attribution

Compare $J(\theta_0, \phi_0)$, $J(\theta_0, \phi_1)$, $J(\theta_1, \phi_0)$, and $J(\theta_1, \phi_1)$ on exactly the same test tasks. Fix environment state distributions and the evaluator; repeat across policy initialization and random seeds. Measure Δ_{int}, not only final-minus-initial gain. Add a matched budget spent on extra rollouts, extra inference, random harness search, and simple prompt tuning. Report both *improvement cost* and *steady-state serving cost*: a costly harness search may still be economical if amortized over sufficient future tasks, but must not be portrayed as cost-free.

18.4 Study D: long-context and memory robustness

Use controlled cases where key information appears early, late, multiple times, in contradiction, or only within a multimodal descriptor. Compare recency, retrieval, compression, and adaptive policies at the same context budget. Require evidence identifiers for selected text and an audit of omitted evidence. Measure source-grounded answer accuracy, fact conflict handling, retention of constraints, privacy-policy compliance, tokens/decision, memory write precision, and recovery after a stale-memory correction. The system should be able to score a low evidence-recall rate as a *harness failure* rather than incorrectly attributing it to the model's reasoning ability.

18.5 Study E: absent, noisy, and delayed reward

Create paired tasks with (i) exact outcome verifiers, (ii) noisy learned judges, (iii) observed delayed feedback with known logging policy in a simulator, and (iv) truly missing feedback. For recommendation-style experiments, distinguish delayed positive feedback from unexposed items and evaluate counterfactual estimators only where logging propensities and positivity assumptions hold. Compare the same learner under correct versus deliberately ambiguous reward provenance. Inspect whether typed uncertainty and censoring reduce inappropriate gradient updates; avoid assuming that a self-judgment can identify correctness on otherwise unverifiable open-world tasks.

18.6 Study F: generalization of skills, context and harness components

Hold out complete task families, not only instances. Use unseen tool schemas, tasks whose best action sequence requires new planning, and a harness with different memory budget. Report absolute task quality, degradation relative to in-domain, and transfer efficiency per additional training sample. If AHC-trained policies transfer no better than framework-native policies under matched data and compute, the hypothesis that standardization *induces model generalization* is unsupported even if portability benefits remain.

18.7 Study G: initialization quality and guarded self-evolution

For each task, compare four initializers: a minimal generic template, an expert-designed template, an LLM-assisted initializer, and an intentionally capability-deficient initialization. Hold the base policy, task access, initial documentation, tool permissions and optimization budget fixed. At the bootstrap

Table 10: Planned evidence matrix. No values are claimed; the table identifies the estimand and primary confounder.

Study	Primary metric	Control / confounder
A: Adaptation	Coverage, time/LoC, conformance	Framework-native reference; developer familiarity
B: Data fidelity	RL-eligible decisions, reproducibility	Same input evidence and training budget
C: Co-optimization	Δ_{int}, success/-cost/safety	Fixed evaluator/test split; matched search budget
D: Long context	Evidence recall and utility per token	Same context budgets and tasks
E: Delayed reward	Calibration, bias and learner stability	Logging policy, delay, censoring, signal reliability
F: Transfer	Held-out family quality and negative transfer	Equal samples/compute; unseen tool schemas

stage measure readiness-gate precision/recall on seeded defects, engineer time, false-positive "ready" declarations, unsupported-capability discovery, and initial task success. Then apply the *same* harness-edit search budget and independent validation protocol to each H_0. Measure outcome versus edit count, total search/serving cost, module churn, rejected unsafe edits, performance on unseen tasks, and whether initialization quality changes the reachable improvement frontier. Add three evolution controls: no edits, random legal edits, and additional inference-time search with no persistent modification. The key falsification criterion is that automatically initialized modular harnesses do *not* improve relative to simple controls once compute, evidence and human effort are matched.

18.8 Minimum statistical and reproducibility requirements

Primary metrics, hypothesis tests, task splits, and stop conditions should be defined before inspecting the final test set. Bootstrap by task/episode, not by individual token; report variability across seeds and environment conditions. Maintain a frozen test set for harness search and a second, independent one for final cross-domain claims. Report failed conformance cases and discarded traces. Public release should include sanitized schema fixtures, adapter code, evaluator definitions, task-environment versions, and a machine-readable run manifest, subject to licensing, confidentiality, and privacy constraints. In environments that cannot be faithfully replayed, mark the limitation explicitly and avoid deterministic counterfactual claims.

19 A Practical Path from Proposal to Community Reference Standard

An industry reference standard is an ecosystem achievement, not a naming decision. We propose four successive milestones. **Milestone 1: shared vocabulary.** Publish a concise ontology separating state, context, action proposal, verified effect, evaluation, memory update, and termination. Cross-map Harness^2's eight editable surfaces and existing SDK abstractions onto it. **Milestone 2: executable conformance.** Release AHC Core and a small suite of normal/error/retry tests with two independent adapters, one based on an event-driven runtime and one on a graph-based runtime. **Milestone 3: learning validity.** Add Training and Replay profiles with verifiable extraction to at least two post-training pipelines and demonstrate correctly rejected invalid trajectories. **Milestone 4: multi-domain reference evaluation.** Publish budget-matched studies of context optimization, policy adaptation, cross-pair interactions, and held-out transfer with negative results included.

Our explicit preference is *standardize the narrow waist*: keep the boundary between policy and environment stable, while allowing internal implementations and task-specific schemas to evolve rapidly. Governance should track schema version negotiation, safe deprecation, privacy and reten-

Table 11: Pre-registrable hypotheses and required tests.

ID	Hypothesis	Test and falsification criterion
H1	Standardized contracts reduce integration work.	Measure integration engineer-hours and failed conformance tests across 3 runtimes; hypothesis fails if savings disappear after implementation overhead.
H2	Better replayability improves RL data validity.	Compare fraction of admissible training transitions, missing masks and reward leakage; hypothesis fails if no quality change.
H3	Co-optimization adds value beyond single-axis changes.	2×2 factorial on matched seeds and compute; fail if Δ_{int} is non-positive or uncertain.
H4	Common lifecycle features improve held-out-domain transfer.	Train with/without standardized metadata using same trajectory information and budget; fail if unseen-domain success does not improve.
H5	Learned context allocation improves reward per token.	Fix the context budget; measure success, evidence recall, hallucinations and token cost; fail if static retrieval dominates.
H6	Guarded releases reduce safety regressions.	Compare unsafe-effect rates and false vetoes on adversarial and routine tasks; fail if safety gain trades off unacceptable utility.

tion, and policy permissions. A standard that freezes the field into eight mandatory files would likely be brittle; a standard that records only free-form text logs would be too weak for algorithmic training validity. The viable middle ground is a versioned execution-and-learning contract with a small normative core and declared extension profiles.

20 Empirical Protocol: Falsifiable Claims

No experiments are reported here. Table 11 defines concrete hypotheses that future implementation should attempt to reject, not assume true.

20.1 Benchmark matrix and controls

Use at least three qualitatively different task families (coding, browser, recommendation or another domain with delayed consequences). Within each family, define disjoint training, validation, and test tasks, including distribution shift and unseen tool schemas. Hold constant model initialization, sampling budget, environment versions, action permissions, evaluation harness, wall-clock budgets, and number of optimization attempts. Include frozen model + default harness, frozen model + optimized harness, fine-tuned model + default harness, and jointly optimized system, with random or compute-matched harness-search baselines. Include a deliberately *misaligned harness* condition and the initialization-quality controls in Study G (Section 18.7) to test missing or inappropriate task capabilities.

20.2 Metrics

Report (a) verified task success with confidence intervals; (b) outcome reliability across seeds and repeated episodes; (c) safety violation and false refusal rates; (d) token, tool, training and human cost; (e) p50/p95 latency, steps-to-completion and timeouts; (f) trace completeness, deterministic replay fidelity and adapter coverage; (g) cross-domain/held-out-tool transfer; and (h) calibration of evaluator decisions against independent ground truth where obtainable. For recommendation tasks, distinguish observational metrics from randomized online effects and disclose exposure/censoring assumptions. Component ablation should include removal of persistent memory, evaluator, skill loading, and trained context selection *at matched budgets*.

20.3 Data, statistical discipline, and release criteria

A minimum credible study should pre-register primary outcomes, run multiple seeds, report confidence intervals with task-level pairing, prevent benchmark-specific prompts from leaking into test tasks, and publish complete version/configuration hashes. Release gates should require non-regression on safety-critical slices and report any exceptions. Human evaluation should be blinded when practicable. A failed hypothesis is valuable evidence about the standardization boundary; negative results should not be suppressed by merging metrics or repeatedly tuning on the test set.

21 Threats to Validity and Failure Modes

Reward misspecification and leakage. A weak judge can reward confident text, excessive tool use, or superficial success. Separating verifier evidence and model-generated claims, and checking against hidden task outcomes, helps but does not eliminate reward hacking.

Context omission and memory contamination. Compression may remove decisive evidence; long-term memory can entrench stale or private information. Provenance, scopes, expiry, write gates, and conflict tests are needed. Long contexts also raise cost and latency constraints.

Harness-induced distribution shift. Search may improve a fixed policy yet harm a newly trained policy; learning under a frozen harness may fail when production interfaces change. Version-aware rollouts and the factorial design provide diagnosis, not an automatic fix.

Incomparable action spaces. A schema match is not behavioral equivalence. Likewise, a complete module manifest does not establish that the initialized harness is capable of a particular task. Particularly with concurrent agents or asynchronous external tools, event order may be partial and rollouts non-replayable. Event causality and replay-mode declarations are mandatory.

Governance and privacy. Trace capture can reveal secrets or sensitive preferences. Minimal logging, redaction, access controls, retention, and user-consent boundaries must be integral to conformance. The existence of a standardized action should never be interpreted as authorization to execute it.

Selection bias in evaluation. Comparisons can become invalid if harness variants change task allocation, refusal rates, observation availability, or evaluation difficulty. Define denominators and measure abstention/deferral explicitly.

Bootstrap and autonomous-edit validity. An LLM-initialized harness can omit necessary actions, fabricate evaluation criteria, or exploit tests used for its own promotion. Report bootstrap false positives, negative-control initializers, and the provenance of seed documents. All automatic edits must remain bounded by independently enforced permissions; gains from weakening a self-generated judge do not establish task improvement.

22 Discussion: Where Standardization Should Stop

The proposed contract is intentionally smaller than the set of algorithms needed for a functioning agent. Standards cannot mandate a universally optimal task initializer, context compressor, reward model, planning algorithm, or memory representation. AHC aims to standardize the ability to *observe, compare, and train* these choices, not to decide every implementation detail. It should adopt or interoperate with existing tool, environment, and telemetry standards rather than replace them [13, 6, 16].

The ambition to encode *any* problem must be restricted to representational compatibility, never conflated with a guarantee that the available action set and observations make that problem solvable. A common event schema may reduce integration cost without improving policy transfer; that negative finding would still be valuable. Conversely, standards that lock in a fixed eight-file topology may block future forms of multimodal sensing, concurrency, or learned memory management. We advocate a stable semantic boundary with explicit bootstrap readiness, versioned module-edit gates, typed domain extensions, and failure reporting. The adoption milestones in Section 19 specify what would be needed before calling this a community reference standard.

23 Conclusion

We propose that an agent harness can be organized into three layers: *stable execution contracts*, *task-specific initial module configurations*, and *guarded feedback-driven revisions*. These layers provide the hypothesized substrate for learnable agent behavior: a reproducible, versioned execution cycle joining model-visible context, policy-selected action, authorized environment effect, observation, evaluation, state update, and termination. The initial harness is a bootstrapped artifact, not a universal task solution; its readiness must be checked, and its evolution must be independently tested. The paper articulates seven explicit design theses, compares current frameworks in their appropriate layers, proposes a testable AHC contract with independent conformance profiles, and defines a controlled agenda for harness optimization, LLM post-training, and their interaction. We make four deliberate distinctions: standardized interfaces are not universal solutions; module editability is not evidence of self-improvement; successful termination is not a ground-truth reward; and joint optimization is not evidence of synergy without controlled cross-pair and budget comparisons. AHC is a candidate reference standard, not an adopted industry standard. Its scientific value depends on whether independent implementations and held-out domain experiments demonstrate measurable improvements in portability, trace validity, learning efficiency, or reliability. These are the next necessary steps.

Acknowledgments and Declaration

This manuscript integrates a researcher-proposed execution-loop thesis, technical literature synthesis, and a candidate reference contract. No new implementation, experiment, or benchmark result is claimed. Bootstrap algorithms, conformance checks, and evolution gates in this revision are proposed designs, not validated implementations. Bibliographic details were checked against publicly accessible source pages as of the date on the title page. The author list, institutional affiliations, funding acknowledgments, disclosure statements, and any empirical claims must be reviewed and supplied by the responsible human authors before dissemination.

References

[1] L. A. Agrawal et al. “GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning”. In: (2025). arXiv: 2507.19457. URL: https://arxiv.org/abs/2507.19457.

[2] M. Cheng et al. “Agent-R1: A Unified and Modular Framework for Agentic Reinforcement Learning”. In: (2025). arXiv: 2511.14460. URL: https://arxiv.org/abs/2511.14460.

[3] Google Cloud. *Agent Development Kit Documentation*. Official framework documentation; accessed October 21, 2026. 2026. URL: https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk.

[4] J. Guo, Z. Hao, C. Wang, et al. “From Question Answering to Task Completion: A Survey on Agent System and Harness Design”. In: (2026). arXiv: 2606.20683. URL: https://arxiv.org/abs/2606.20683.

[5] P. Hebbar et al. "SIA: Self Improving AI with Harness & Weight Updates". In: (2026). arXiv: 2605.27276. URL: https://arxiv.org/abs/2605.27276.

[6] Hugging Face. *OpenEnv: Agentic Execution Environments*. Project documentation; accessed October 21, 2026. 2026. URL: https://huggingface.co/openenv.

[7] C. E. Jimenez, J. Yang, A. Wettig, et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" In: (2023). arXiv: 2310.06770. URL: https://arxiv.org/abs/2310.06770.

[8] O. Khattab, A. Singhvi, P. Maheshwari, et al. "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines". In: (2023). arXiv: 2310.03714. URL: https://arxiv.org/abs/2310.03714.

[9] LangChain. *LangGraph Persistence and Checkpointing*. Official framework documentation; accessed October 21, 2026. 2026. URL: https://docs.langchain.com/oss/python/langgraph/persistence.

[10] J. Lin et al. "Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses". In: (2026). arXiv: 2604.25850. URL: https://arxiv.org/abs/2604.25850.

[11] X. Luo et al. "Agent Lightning: Train ANY AI Agents with Reinforcement Learning". In: (2025). arXiv: 2508.03680. URL: https://arxiv.org/abs/2508.03680.

[12] Q. Mao, W. Qu, D. Guo, et al. "SafeEvolve: Harness-Policy Co-Evolution from Agent Experience for Safety Alignment". In: (2026). arXiv: 2609.02786. URL: https://arxiv.org/abs/2609.02786.

[13] Model Context Protocol Contributors. *Model Context Protocol Specification, 2026-07-28*. Protocol specification; accessed October 21, 2026. 2026. URL: https://modelcontextprotocol.io/specification/2026-07-28.

[14] X. Ning, K. Tieu, D. Fu, et al. "Code as Agent Harness". In: (2026). arXiv: 2605.18747. URL: https://arxiv.org/abs/2605.18747.

[15] OpenAI. *OpenAI Agents SDK Documentation*. Official SDK documentation; accessed October 21, 2026. 2026. URL: https://openai.github.io/openai-agents-python/.

[16] OpenTelemetry Contributors. *OpenTelemetry GenAI Semantic Conventions*. Evolving specification; accessed October 21, 2026. 2026. URL: https://github.com/open-telemetry/semantic-conventions-genai.

[17] C. Packer et al. "MemGPT: Towards LLMs as Operating Systems". In: (2023). arXiv: 2310.08560. URL: https://arxiv.org/abs/2310.08560.

[18] Z. Shao et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: (2024). arXiv: 2402.03300. URL: https://arxiv.org/abs/2402.03300.

[19] Y. Wang, H. Zhu, Z. Hu, Y. Yuan, Z. Chen, et al. "Rethinking the Evaluation of Harness Evolution for Agents". In: (2026). arXiv: 2607.12227. URL: https://arxiv.org/abs/2607.12227.

[20] Z. Wei et al. "Self-Evolving Coding Agents via Harness-Policy Optimization". In: (2026). arXiv: 2610.22791. URL: https://arxiv.org/abs/2610.22791.

[21] Z. Xi, Y. Ding, W. Chen, et al. "AgentGym: Evolving Large Language Model-based Agents across Diverse Environments". In: (2024). arXiv: 2406.04151. URL: https://arxiv.org/abs/2406.04151.

[22] R. Xu et al. "Harness^2: Recursive Agent Harnessing for an Open World". In: (2026). arXiv: 2610.14224. URL: https://arxiv.org/abs/2610.14224.

[23] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. "Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains". In: (2024). arXiv: 2406.12045. URL: https://arxiv.org/abs/2406.12045.

[24] S. Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: (2022). arXiv: 2210.03629. URL: https://arxiv.org/abs/2210.03629.

[25] Z. Yu et al. “Recursive Experiential–Working Memory Evolution for Long-Horizon Agent Harnesses”. In: (2026). arXiv: 2608.24876. URL: https://arxiv.org/abs/2608.24876.

[26] M. Yuksekgonul et al. “TextGrad: Automatic “Differentiation” via Text”. In: (2024). arXiv: 2406.07496. URL: https://arxiv.org/abs/2406.07496.

[27] L. Zhang, R. Zhou, D. Song, Z. Chen, Y. Tian, et al. “HarnessCompass: Guiding Automatic Harness Evolution toward Generalizable and Effective Agent Harnesses”. In: (2026). arXiv: 2608.01918. URL: https://arxiv.org/abs/2608.01918.

A Illustrative Typed Record (Non-normative)

The following pseudocode is a conceptual schema rather than a finalized wire specification. Extensions can hold multi-agent causality, permission receipts, and delayed feedback IDs.

```
RunHeader {
  run_id: string; task_id: string; environment_version: string;
  policy_checkpoint: string; harness_hash: string;
  task_spec_id: string; initializer_id?: string;
  initialization_manifest_ref?: secure_uri_or_hash;
  schema_version: string; replay_mode: enum;
}
StepEvent {
  run_id: string; step_id: string; parent_step_id?: string;
  type: enum{context, model_decision, action_validation,
            execution, observation, evaluation,
            state_update, termination};
  timestamp: time; payload_ref: secure_uri_or_hash;
  provenance: {producer_id, component_version};
}
DecisionRecord {
  context_ref: secure_uri_or_hash;
  available_action_schemas: [schema_id];
  allowed_action_mask?: binary_or_set;
  proposed_action: typed_value;
  behavior_policy_logprob?: number;
}
OutcomeRecord {
  executed_action?: typed_value;
  action_status: enum{allowed, denied, timeout,
                      partial_commit, error, committed};
  observation_ref?: secure_uri_or_hash;
  verification_status?: enum{unknown, pass, fail};
  reward?: {value: number, source: string,
            maturity: enum{immediate, delayed, provisional}};
  terminal?: {done: bool, reason: string};
}
MemoryDelta {
  scope: enum{working, persistent};
  owner: string; changed_refs: [secure_uri_or_hash];
  evidence_refs: [secure_uri_or_hash];
  consent_or_policy_id?: string; expiry?: time;
}
```

B Illustrative Bootstrap and Evolution Manifest

```
TaskHarnessManifest {
  contract_version: string;
```

```
  task_spec_id: string; environment_ref: string;
  selected_modules: [{interface_id, implementation_id,
                      version, config_ref, optional: bool}];
  initializer_provenance: {type, seed_refs, model_id?};
  bootstrap_report: {schema_ok, executable_path_ok,
                     evidence_ok, containment_ok,
                     validation_protocol_ok};
  allowed_edit_surfaces: [interface_id];
  immutable_permission_policy_ref: secure_uri_or_hash;
}
HarnessRevision {
  parent_harness_hash: string; candidate_harness_hash: string;
  changed_modules: [interface_id]; cause_trace_refs: [string];
  diagnostic_hypotheses: [string]; validation_protocol_id: string;
  independent_score_refs: [string]; safety_result: string;
  promotion: enum{accepted, rejected, human_review};
}
```

C Reference Conformance Test Cases

1. **Denied tool call**: Model proposes restricted action; validator vetoes; no environment side effect is recorded; trajectory includes attempted action and reason.
2. **Timeout with unknown commit**: Tool fails after request transmission; harness records uncertain effect and does not blindly retry a non-idempotent request.
3. **Reward later than termination**: Session terminates after recommendation serving; deferred engagement attaches to original run with observation-policy provenance.
4. **Context replacement**: Context builder compresses old history; retained citation/evidence references are recorded for replay and omission analysis.
5. **Nested subagent**: Parent trace records delegation, child trace/context isolation, and return value; events use causal links rather than invented chronological linearity.
6. **Harness update**: Two identical model checkpoints use different context budgets; the difference is auditable in the run headers and decision records.
7. **Policy update**: Two model checkpoints use the same immutable harness; the runtime records the sampler and log-probability availability before any RL computation.
8. **Missing initialization capability**: An initializer omits a required task action; the readiness report fails instead of declaring the agent "self-improving."
9. **Untrusted evaluator**: A task has no validated feedback; a proposed harness edit must not be promoted based solely on a self-generated reward.
10. **Memory versus policy change**: An authorized memory write does not change the harness version; a new memory retrieval rule creates a candidate revision and requires a promotion test.
11. **Unauthorized self-modification**: A proposed harness edit widens tool permissions; the governance layer rejects the edit regardless of claimed task benefit.

D Proposed Implementation Sequence

1. Build a minimal sequential executor with immutable run IDs, typed action validation, terminal reasons and a recorder; no automatic optimization.
2. Define a task declaration and initialize the minimum required modules for code and browser adapters; pass AHC-C and AHC-I bootstrap tests.
3. Add one environment adapter each for code and browser workflows; pass AHC-C conformance tests.
4. Add persistent memory with provenance and guarded write semantics; test retrieval omission, conflicting facts, and data expiry.
5. Add observation/reward connectors, explicit credit-assignment metadata, and offline SFT/RL extraction; validate a tiny synthetic environment before expensive training.

6. Add a frozen-policy harness optimization stage and a frozen-harness policy update stage; evaluate four pairings and rollback failures.
7. Expand to delayed-feedback tasks, held-out tool families, and new domains; report cost-normalized transfer, not only in-domain success.